

SOLIDWORKS API (.NET VBA C#) — Examples (47/50): Set_Body_for_View_Example_CSharp Set_View_Scale_Opposite_Parent_View_Scal

SOLIDWORKS API Documentation (converted from CHM)

Set Body for View Example (C#)

This example shows how to show just one body of a multibody part in a drawing view.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\multibody\multi_inter.sldprt.
// 2. Save the part document as a drawing document:
//   a. Click File > Make Drawing from Part.
//   b. Click OK on the Sheet Format/Size dialog.
//   c. Drag the *Isometric view from the View Palette onto
//      the drawing sheet.
// 3. Select the drawing view.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Shows one body of the multibody part
//    in the drawing view.
// 2. Examine the drawing and the Immediate window.
//
// NOTE: Because the part document is used elsewhere, do not save
// changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;
using System.Windows.Forms;
namespace BodiesViewCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SolidWorks.Interop.sldworks.View swView = default(SolidWorks.Interop.sldworks.View);
            int nbrBodies = 0;
            object[] arrBody = null;
            Body2 swBody = default(Body2);
            Face2 swFace = default(Face2);
            Entity swEnt = default(Entity);
            SelectData swSelData = default(SelectData);
            PartDoc swPart = default(PartDoc);
            bool status = false;
            DispatcherWrapper[] arrBodiesIn = new DispatcherWrapper[1];
            object[] Bodies = new object[1];
            int i = 0;
            int objType = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swView = (SolidWorks.Interop.sldworks.View)swSelMgr.GetSelectedObject6(1, -1);
            if ((swView == null))
            {
                MessageBox.Show("View not selected.");
                return;
            }
            nbrBodies = swView.GetBodiesCount();
            Debug.Print("Number of bodies: " + nbrBodies);
            if ((nbrBodies < 1))
            {
                MessageBox.Show("No bodies in selected view.");
                return;
            }
        }
    }
}
```

```

arrBody = (object[])swView.Bodies;
for (i = 0; i < arrBody.Length; i++)
{
    swBody = (Body2)arrBody[i];
    swSelData = (SelectData)swSelMgr.CreateSelectData();
    swSelData.View = swView;
    status = swBody.Select2(false, swSelData);
    // Object type 76 is a solid body
    objType = swSelMgr.GetSelectedObjectType3(1, -1);
    if ((objType == 76))
    {
        Debug.Print(" Object type: solid body");
    }
    if (((!(int)swSelectType_e.swSelSOLIDBODIES == swSelMgr.GetSelectedObjectType3(1, -1))))
    {
        MessageBox.Show("Solid body not found.");
    }
    swFace = (Face2)swBody.GetFirstFace();
    while ((swFace != null))
    {
        swEnt = (Entity)swFace;
        // Select using IEntity
        status = swEnt.Select4(true, swSelData);
        Debug.Assert(status);
        swFace = (Face2)swFace.GetNextFace();
    }
    Debug.Print(" Name of body: " + swBody.GetSelectionId());
}

swModel.ClearSelection2(true);

// Get the bodies from referenced model
swModel = (ModelDoc2)swView.ReferencedDocument;
swPart = (PartDoc)swModel;
arrBody = (object[])swPart.GetBodies2((int)swBodyType_e.swSolidBody, true);
if ((nbrBodies == 1))
{
    swView.Bodies = (arrBody);
}
else
{
    // Set the body to include in the drawing view
    Bodies[0] = arrBody[0];
    arrBodiesIn[0] = new DispatcherWrapper(Bodies[0]);
    swView.Bodies = (arrBodiesIn);
}

swModel.ClearSelection2(true);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Set Body for View Example (VBA)

This example shows how to show just one body of a multibody part in a drawing view.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi_inter.sldprt.
' 2. Save the part document as a drawing document:
'   a. Click File > Make Drawing from Part.
'   b. Click OK on the Sheet Format/Size dialog.
'   c. Drag the *Isometric view from the View Palette onto
'      the drawing sheet.
' 3. Select the drawing view.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Shows one body of the multibody part
'    in the drawing view.
' 2. Examine the drawing and the Immediate window.
'
' NOTE: Because the part document is used elsewhere, do not save
' changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swView As SldWorks.View
Dim nbrBodies As Long
Dim arrBody As Variant
Dim swBody As SldWorks.Body2
Dim swFace As SldWorks.Face2
Dim swEnt As SldWorks.Entity
Dim swSelData As SldWorks.SelectData
Dim bool As Boolean
Dim arrBodiesIn As Variant
Dim Bodies(0) As Object
Dim i As Long
Dim objType As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)
    If (swView Is Nothing) Then
        MsgBox "View not selected."
        Exit Sub
    End If
    nbrBodies = swView.GetBodiesCount
    Debug.Print "Number of bodies: " & nbrBodies
    If (nbrBodies < 1) Then
        MsgBox "No bodies in selected view."
        Exit Sub
    End If
    arrBody = swView.Bodies
    For i = 0 To UBound(arrBody)
        Set swBody = arrBody(i)
        Set swSelData = swSelMgr.CreateSelectData
        swSelData.View = swView
        bool = swBody.Select2(False, swSelData)
        ' Object type 76 is a solid body
        objType = swSelMgr.GetSelectedObjectType3(1, -1)
        If (objType = 76) Then

```

```

        Debug.Print " Object type: solid body"
    End If
    If (Not (swSelSOLIDBODIES = swSelMgr.GetSelectedObjectType3(1, -1))) Then
        MsgBox "Solid body not found."
    End If
    Set swFace = swBody.GetFirstFace
    Do While Not swFace Is Nothing
        Set swEnt = swFace
        ' Select using IEntity
        bool = swEnt.Select4(True, swSelData): Debug.Assert bool
        Set swFace = swFace.GetNextFace
    Loop
    Debug.Print "   Name of body: " & swBody.GetSelectionId
Next i

```

```
swModel.ClearSelection2 True
```

```

' Get the bodies from referenced model
Set swModel = swView.ReferencedDocument
arrBody = swModel.GetBodies2(swSolidBody, True)
If (nbrBodies = 1) Then
    swView.Bodies = (arrBody)
Else
    ' Set the body to view
    Set Bodies(0) = arrBody(0)
    arrBodiesIn = Bodies
    swView.Bodies = (arrBodiesIn)
End If

```

```
swModel.ClearSelection2 True
```

```
End Sub
```

Set Body for View Example (VB.NET)

This example shows how to show just one body of a multibody part in a drawing view.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi_inter.sldprt.
' 2. Save the part document as a drawing document:
'   a. Click File > Make Drawing from Part.
'   b. Click OK on the Sheet Format/Size dialog.
'   c. Drag the *Isometric view from the View Palette onto
'       the drawing sheet.
' 3. Select the drawing view.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Shows one body of the multibody part
'    in the drawing view.
' 2. Examine the drawing and the Immediate window.
'
' NOTE: Because the part document is used elsewhere, do not save
'       changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports System.Runtime.InteropServices
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swView As View
        Dim nbrBodies As Long
        Dim arrBody As Object
        Dim swBody As Body2
        Dim swFace As Face2
        Dim swEnt As Entity
        Dim swSelData As SelectData
        Dim bool As Boolean
        Dim arrBodiesIn(0) As DispatchWrapper
        Dim Bodies(0) As Object
        Dim i As Long
        Dim objType As Long

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swView = swSelMgr.GetSelectedObject6(1, -1)
        If (swView Is Nothing) Then
            MsgBox("View not selected.")
            Exit Sub
        End If

        nbrBodies = swView.GetBodiesCount
        Debug.Print("Number of bodies: " & nbrBodies)
        If (nbrBodies < 1) Then
            MsgBox("No bodies in selected view.")
            Exit Sub
        End If

        arrBody = swView.Bodies
        For i = 0 To UBound(arrBody)
            swBody = arrBody(i)
            swSelData = swSelMgr.CreateSelectData
            swSelData.View = swView
```

```

        bool = swBody.Select2(False, swSelData)
        ' Object type 76 is a solid body
        objType = swSelMgr.GetSelectedObjectType3(1, -1)
        If (objType = 76) Then
            Debug.Print(" Object type: solid body")
        End If
        If (Not (swSelectType_e.swSelSOLIDBODIES = swSelMgr.GetSelectedObjectType3(1, -1))) Then
            MsgBox("Solid body not found.")
        End If
        swFace = swBody.GetFirstFace
        Do While Not swFace Is Nothing
            swEnt = swFace
            ' Select using IEntity
            bool = swEnt.Select4(True, swSelData) : Debug.Assert(bool)
            swFace = swFace.GetNextFace
        Loop
        Debug.Print("   Name of body: " & swBody.GetSelectionId)
    Next i

    swModel.ClearSelection2(True)

    ' Get the bodies from referenced model
    swModel = swView.ReferencedDocument
    arrBody = swModel.GetBodies2(swBodyType_e.swSolidBody, True)
    If (nbrBodies = 1) Then
        swView.Bodies = (arrBody)
    Else
        ' Set the body to include in the drawing view
        Bodies(0) = arrBody(0)
        arrBodiesIn(0) = New Dispatcher(Bodies(0))
        swView.Bodies = (arrBodiesIn)
    End If

    swModel.ClearSelection2(True)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Set BOM Balloon Text Example (VBA)

This example shows how to set the text in a BOM balloon with custom text.

```

'-----
' Preconditions:
' 1. Open a drawing document with a split-circle BOM balloon
'   with custom text in the upper and lower portions of
'   of the balloon.
' 2. Select the balloon specified in step 1.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Appends new text to the existing text in the selected balloon.
' 2. Examine the selected balloon in the drawing and the
'   the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swSelMgr             As SldWorks.SelectionMgr
    Dim swNote               As SldWorks.Note
    Dim swAnn                As SldWorks.Annotation
    Dim bRet                 As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swNote = swSelMgr.GetSelectedObject6(1, -1)
    Set swAnn = swNote.GetAnnotation
    Debug.Assert swNote.IsBomBalloon
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print "  Name = " & swAnn.GetName
    Debug.Print "   Upper text = " & swNote.GetBomBalloonText(True) & " [" & swNote.GetBomBalloonTextStyle(True) & "]"
    Debug.Print "   Lower text = " & swNote.GetBomBalloonText(False) & " [" & swNote.GetBomBalloonTextStyle(False) & "]"

    ' Append new text to existing text if balloon has custom text
    bRet = swNote.SetBomBalloonText(swNote.GetBomBalloonTextStyle(True), swNote.GetBomBalloonText(True) & " Upper", swNote.GetBomBalloonTextStyle(False),
    swNote.GetBomBalloonText(False) & " Lower")

End Sub

```

Set Component State Example (VBA)

This example shows how to set the suppression state of the selected assembly components.

```
'-----  
' Preconditions:  
' 1. Open an assembly.  
' 2. Select one or more components.  
'  
' Postconditions:  
' 1. Sets the states of the selected components to  
'    suppressed.  
' 2. Examine the graphics area and FeatureManager  
'    design tree.  
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swAssy As SldWorks.AssemblyDoc  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swConfigMgr As SldWorks.ConfigurationManager  
Dim swConf As SldWorks.Configuration  
Dim i As Long  
Dim nSelCount As Long  
Dim swCompArr() As SldWorks.Component2  
Dim vCompArr As Variant  
Dim sConfName As String  
Dim bRet As Boolean
```

```
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swAssy = swModel  
Set swSelMgr = swModel.SelectionManager  
Set swConfigMgr = swModel.ConfigurationManager  
Set swConf = swConfigMgr.ActiveConfiguration
```

```
sConfName = swConf.Name
```

```
ReDim swCompArr(0)
```

```
nSelCount = swSelMgr.GetSelectedObjectCount2(-1)
```

```
For i = 1 To nSelCount
```

```
    If Not swSelMgr.GetSelectedObjectsComponent2(i) Is Nothing Then
```

```
        Set swCompArr(i - 1) = swSelMgr.GetSelectedObjectsComponent4(i, -1)
```

```
        ReDim Preserve swCompArr(UBound(swCompArr) + 1)
```

```
    End If
```

```
Next i
```

```
Debug.Assert UBound(swCompArr) > 0
```

```
ReDim Preserve swCompArr(UBound(swCompArr) - 1)
```

```
vCompArr = swCompArr
```

```
bRet = swAssy.SetComponentState(swComponentSuppressed, (vCompArr), swThisConfiguration, sConfName, False): Debug.Assert bRet
```

End Sub

Set Components and Transforms for Interference Detection Example (C#)

This example shows how to:

- move an assembly component to an interfering position by replacing its transform.
- detect interference.

This example also shows how to apply absolute transforms to all components in order to multiply by identity transforms, which results in the components remaining in their current positions during interference detection.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly exists.
// 2. Copy and paste Main into SolidWorksMacro.cs of your macro.
// 3. Click Project > Add Class and copy and paste Class into Class1.cs.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly.
// 2. Gets the components of the assembly and their transforms.
// 3. Creates a new transform for Part1^AssemInterferenceDetection-2.
// 4. Sets the components and their transforms, using the existing
//    rotation and the translation from the assembly origin.
//    a. Gets the number of interferences.
//    b. Examine the graphics area at the System.Diagnostics.Debugger.Break()
//       statement to visually verify the interferences, then press F5.
//    c. Programmatically verifies that the number of interferences
//       is as expected.
//    d. Gets whether:
//       * each returned interference is correct.
//       * the components remained in their starting positions during
//         interference detection.
//       * the components are at their starting positions.
// 5. Examine the Immediate window and graphics area.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
//Main
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Collections;
using System.Collections.Generic;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ArrayList colStartingPositions = new ArrayList();

        public void Main()
        {
            Class1 Class1 = new Class1();
            Class1 = new Class1();

            int expectedNumberOfInterferences = 0;
            expectedNumberOfInterferences = 2;

            ArrayList expectedInterferenceVolumes = default(ArrayList);
            expectedInterferenceVolumes = new ArrayList();
            expectedInterferenceVolumes.Add(0.00);
            expectedInterferenceVolumes.Add(0.01);

            ArrayList expectedNumberOfInterferenceComps = default(ArrayList);
            expectedNumberOfInterferenceComps = new ArrayList();
            expectedNumberOfInterferenceComps.Add(2);
            expectedNumberOfInterferenceComps.Add(2);

            bool bResult = false;
            bResult = true;
            AssemblyDoc swAssem = default(AssemblyDoc);
            swAssem = (AssemblyDoc)OpenAssembly();

            RecordStartingPositions(swAssem);

            SelectAllComponents(swAssem);

            InterferenceDetectionMgr swInfrMgr = default(InterferenceDetectionMgr);
            swInfrMgr = (InterferenceDetectionMgr)swAssem.InterferenceDetectionManager;

            swInfrMgr.UseTransform = true;

            Component2[] swComps = new Component2[3];
            swComps[0] = (Component2)swAssem.GetComponentByName("Part1^AssemInterferenceDetection-1");
            swComps[1] = (Component2)swAssem.GetComponentByName("Part1^AssemInterferenceDetection-2");
            swComps[2] = (Component2)swAssem.GetComponentByName("Part1^AssemInterferenceDetection-3");
            MathTransform[] swXfms = new MathTransform[3];
            swXfms[0] = (MathTransform)swComps[0].Transform2;
            swXfms[1] = (MathTransform)swComps[1].Transform2;
            swXfms[2] = (MathTransform)swComps[2].Transform2;

            // Replacement transform for Part1^AssemInterferenceDetection-2
            double[] dxfm = new double[16];

            // Use existing rotation
            double[] vxfmCurrent = null;
            vxfmCurrent = (double[])swXfms[1].ArrayData;
            dxfm[0] = (double)vxfmCurrent[0];
            dxfm[1] = (double)vxfmCurrent[1];
            dxfm[2] = (double)vxfmCurrent[2];
            dxfm[3] = (double)vxfmCurrent[3];
            dxfm[4] = (double)vxfmCurrent[4];
            dxfm[5] = (double)vxfmCurrent[5];
        }
    }
}
```

```

dxfm[6] = (double)vXfmCurrent[6];
dxfm[7] = (double)vXfmCurrent[7];
dxfm[8] = (double)vXfmCurrent[8];
// Translation from assembly origin
dxfm[9] = (double)0.9;
dxfm[10] = (double)0.1;
dxfm[11] = (double)0;
dxfm[12] = (double)1;
dxfm[13] = (double)0;
dxfm[14] = (double)0;
dxfm[15] = (double)0;
dxfm[1] = (double)0;
dxfm[2] = (double)0;

MathUtility swMath = default(MathUtility);
swMath = (MathUtility)swApp.GetMathUtility();
swXfms[1] = (MathTransform)swMath.CreateTransform(dxfm);

int lResult = 0;
lResult = swInfrMgr.SetComponentsAndTransforms(swComps, swXfms);

Debug.Print("Transforms applied:");
if (lResult == (int)swSetComponentsAndTransformsStatus_e.swSetComponentsAndTransforms_Succeeded)
{
    Debug.Print(" True");
}
else
{
    Debug.Print(" False: " + lResult + " vs. " + (int)swSetComponentsAndTransformsStatus_e.swSetComponentsAndTransforms_Succeeded);
    bResult = false;
}

int actualNumberOfInterferences = 0;
actualNumberOfInterferences = swInfrMgr.GetInterferenceCount();
Debug.Print("Number of interferences: ");
Debug.Print(" " + actualNumberOfInterferences);

System.Diagnostics.Debugger.Break();
// Examine the graphics area to verify
// the interferences, then press F5

bool bCorrectNumberOfInterferences = false;
bCorrectNumberOfInterferences = expectedNumberOfInterferences == actualNumberOfInterferences;
Debug.Print("Correct number of interferences:");
if (bCorrectNumberOfInterferences)
{
    Debug.Print(" True");
}
else
{
    Debug.Print(" False: " + actualNumberOfInterferences + " vs. " + expectedNumberOfInterferences);
    bResult = false;
}

bool bCorrectInterferences = false;
object[] interferences;
interferences = (object[])swInfrMgr.GetInterferences();
bCorrectInterferences = VerifyInterferencesAgainstExpectation(interferences, expectedInterferenceVolumes, expectedNumberOfInterferenceComps);
Debug.Print("Correct interferences:");
if (bCorrectInterferences)
{
    Debug.Print(" True");
}
else
{
    Debug.Print(" False");
    bResult = false;
}
swInfrMgr.Done();

bool bBackToStartingPositions = false;
bBackToStartingPositions = VerifyFinishingPositions();
Debug.Print("Components back at starting positions:");
if (bBackToStartingPositions)
{
    Debug.Print(" True");
}
else
{
    Debug.Print(" False");
    bResult = false;
}
}

}

public AssemblyDoc OpenAssembly()
{
    swApp.CloseAllDocuments(true);
    DocumentSpecification docspec = default(DocumentSpecification);
    string assemblyFile = null;
    assemblyFile = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\AssemInterferenceDetection.sldasm";
    docspec = (DocumentSpecification)swApp.OpenDocSpec(assemblyFile);
    ModelDoc2 swModel = default(ModelDoc2);
    swModel = (ModelDoc2)swApp.ActiveDoc;
    swModel = (ModelDoc2)swApp.OpenDoc7(docspec);
    AssemblyDoc swAssem = default(AssemblyDoc);
    swAssem = (AssemblyDoc)swModel;
    return swAssem;
}

public void RecordStartingPositions(AssemblyDoc swAssem)
{
    colStartingPositions = new ArrayList();
    object[] vComponents = null;
    vComponents = (object[])swAssem.GetComponents(true);
    if ((vComponents != null))
    {
        foreach (object oneComp in vComponents)
        {
            Component2 swComp = default(Component2);
            swComp = (Component2)oneComp;
            Class1 position = default(Class1);
            position = new Class1();

```

```

        position.RecordReferencePosition(swComp);
        colStartingPositions.Add(position);
    }
}
}
public bool VerifyFinishingPositions()
{
    bool bResult = false;
    bResult = true;

    Debug.Print("Position of: ");
    foreach (Class1 position in colStartingPositions)
    {
        if (position.IsAtReferencePosition())
        {
            Debug.Print(" " + position.Name + " has not moved");
        }
        else
        {
            Debug.Print(" " + position.Name + " has moved");
            bResult = false;
        }
    }
    return bResult;
}
public bool VerifyInterferencesAgainstExpectation(object[] vInterferences, System.Collections.ArrayList expectedVolumes, System.Collections.ArrayL
ist expectedCompCounts)
{
    bool bResult = false;
    bResult = true;
    if ((vInterferences == null))
    {
        Debug.Print("IInterferenceDetectionMgr::GetInterferences returned:");
        if (expectedVolumes.Count == 0)
        {
            Debug.Print(" Null as expected");
        }
        else
        {
            Debug.Print(" Null unexpectedly");
            bResult = false;
        }
    }
    else
    {
        int numInterferences = 0;
        numInterferences = vInterferences.GetUpperBound(0) - vInterferences.GetLowerBound(0) + 1;
        if (numInterferences != expectedVolumes.Count)
        {
            Debug.Print("IInterferenceDetectionMgr::GetInterferences returned unexpected number of interferences: " + numInterferences + " vs. " +
expectedVolumes.Count);
            bResult = false;
        }
        // Cannot assume the same order
        // Try to find matches for each returned interference
        int i = 0;

        Debug.Print("Match found for calculated interference:");
        for (i = vInterferences.GetLowerBound(0); i <= vInterferences.GetUpperBound(0); i++)
        {
            Interference swInterference = default(Interference);
            swInterference = (Interference)vInterferences[i];
            double volume = 0;
            volume = swInterference.Volume;
            int compCount = 0;
            compCount = swInterference.GetComponentCount();
            bool bFoundMatch = false;
            bFoundMatch = false;
            int j;
            for (j = 0; j < (expectedVolumes.Count); j++)
            {
                double expVol;
                expVol = (double)expectedVolumes[j];
                double expCompCounts;
                expCompCounts = (int)expectedCompCounts[j];
                if (Class1.EqualWithinTolerance(expVol, volume) & (expCompCounts == compCount))
                {
                    bFoundMatch = true;
                    expectedVolumes.RemoveAt(j);
                    expectedCompCounts.RemoveAt(j);
                    break;
                }
            }
            if (bFoundMatch)
            {
                Debug.Print(" " + i + ": Volume = " + volume.ToString("0.000000") + " & Component count = " + compCount);
            }
            else
            {
                Debug.Print(" " + i + ": Volume = " + volume.ToString("0.000000") + " & Component count = " + compCount);
            }
            bResult = false;
        }
    }
    int k;
    if (expectedVolumes.Count > 0)
    {
        for (k = 0; k < (expectedVolumes.Count); k++)
        {
            double expVolumes;
            expVolumes = (double)expectedVolumes[k];
            Debug.Print("No match found for expected interference: " + k + ": Volume = " + (expVolumes.ToString("0.000000")) + " & Component co
unt = " + expectedCompCounts[k]);
        }
    }
    return bResult;
}
public void SelectAllComponents(AssemblyDoc swAssem)

```

```

{
    ModelDoc2 swModel = default(ModelDoc2);
    swModel = (ModelDoc2)swAssem;
    swModel.ClearSelection2(true);

    object[] vComponents = null;
    vComponents = (object[])swAssem.GetComponents(true);
    if ((vComponents != null))
    {
        foreach (object vComp in vComponents)
        {
            Component2 swComp = default(Component2);
            swComp = (Component2)vComp;
            swComp.Select4(true, null, false);
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
}

```

[Back to top](#)

//Class

```

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using Macro1CSharp;

public class Class1
{
    private Component2 swComp;
    private MathTransform swReferenceTransform;
    public void RecordReferencePosition(Component2 swComponent)
    {
        swComp = (Component2)swComponent;
        swReferenceTransform = (MathTransform)swComp.Transform2;
    }
    public bool IsAtReferencePosition()
    {
        MathTransform swCurrentTransform = default(MathTransform);
        swCurrentTransform = (MathTransform)swComp.Transform2;

        MathTransform swProduct = default(MathTransform);
        swProduct = (MathTransform)swReferenceTransform.Multiply(swCurrentTransform.Inverse());

        double[] vProduct = null;
        vProduct = (double[])swProduct.ArrayData;
        double product0 = (double)vProduct[0];
        double product1 = (double)vProduct[1];
        double product2 = (double)vProduct[2];
        double product3 = (double)vProduct[3];
        double product4 = (double)vProduct[4];
        double product5 = (double)vProduct[5];
        double product6 = (double)vProduct[6];
        double product7 = (double)vProduct[7];
        double product8 = (double)vProduct[8];
        double product9 = (double)vProduct[9];
        double product10 = (double)vProduct[10];
        double product11 = (double)vProduct[11];
        double product12 = (double)vProduct[12];

        // Require identity transform
        bool bResult = false;
        bResult = EqualWithinTolerance(product0, 1);
        bResult = EqualWithinTolerance(product1, 0) & bResult;
        bResult = EqualWithinTolerance(product2, 0) & bResult;

        bResult = EqualWithinTolerance(product3, 0) & bResult;
        bResult = EqualWithinTolerance(product4, 1) & bResult;
        bResult = EqualWithinTolerance(product5, 0) & bResult;

        bResult = EqualWithinTolerance(product6, 0) & bResult;
        bResult = EqualWithinTolerance(product7, 0) & bResult;
        bResult = EqualWithinTolerance(product8, 1) & bResult;

        bResult = EqualWithinTolerance(product9, 0) & bResult;
        bResult = EqualWithinTolerance(product10, 0) & bResult;
        bResult = EqualWithinTolerance(product11, 0) & bResult;

        bResult = EqualWithinTolerance(product12, 1) & bResult;

        if (!bResult)
        {
            Debug.Print(Name + ":");
            Debug.Print(product0.ToString("0.000000") + "\t" + product1.ToString("0.000000") + "\t" + product2.ToString("0.000000"));
            Debug.Print(product3.ToString("0.000000") + "\t" + product4.ToString("0.000000") + "\t" + product5.ToString("0.000000"));
            Debug.Print(product6.ToString("0.000000") + "\t" + product7.ToString("0.000000") + "\t" + product8.ToString("0.000000"));
            Debug.Print(product9.ToString("0.000000") + "\t" + product10.ToString("0.000000") + "\t" + product11.ToString("0.000000"));
            Debug.Print(product12.ToString("0.000000"));
        }

        return bResult;
    }
}

```

```
}  
public string Name  
{  
    get { return swComp.Name2; }  
}  
  
public static bool EqualWithinTolerance(double A, double B)  
{  
    const double tolerance = 1E-05;  
    double difference = 0;  
    difference = Math.Abs(A - B);  
    return difference < tolerance;  
}  
}
```

[Back to top](#)

Set Components and Transforms for Interference Detection Example (VBA)

This example shows how to:

- move an assembly component to an interfering position by replacing its transform.
- detect interference.

This example also shows how to apply absolute transforms to all components in order to multiply by identity transforms, which results in the components remaining in their current positions during interference detection.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Copy and paste Main into the main module of your macro.
' 3. Click Insert > Class Module and copy and paste Class1 into that
'    class module.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly.
' 2. Gets the components of the assembly and their transforms.
' 3. Creates a new transform for Part1^AssemInterferenceDetection-2.
' 4. Sets the components and their transforms, using the existing
'    rotation and the translation from the assembly origin.
'    a. Gets the number of interferences.
'    b. Examine the graphics area at the Stop statement to visually
'       verify the interferences, then press F5.
'    c. Programmatically verifies that the number of interferences
'       is as expected.
'    d. Gets whether:
'       * each returned interference is correct.
'       * the components remained in their starting positions during
'         interference detection.
'       * the components are at their starting positions.
' 5. Examine the Immediate window and graphics area.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
' Main

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim colStartingPositions As Collection

```

```

Sub main()

```

```

    Dim expectedNumberOfInterferences As Long
    expectedNumberOfInterferences = 2

```

```

    Dim expectedInterferenceVolumes As Collection
    Set expectedInterferenceVolumes = New Collection
    expectedInterferenceVolumes.Add 0.09
    expectedInterferenceVolumes.Add 0.01

```

```

    Dim expectedNumberOfInterferenceComps As Collection
    Set expectedNumberOfInterferenceComps = New Collection
    expectedNumberOfInterferenceComps.Add 2
    expectedNumberOfInterferenceComps.Add 2

```

```

    Dim bResult As Boolean
    bResult = True

```

```

    Dim swAssem As SldWorks.AssemblyDoc
    Set swAssem = OpenAssembly

```

```

    RecordStartingPositions swAssem

```

```

    SelectAllComponents swAssem

```

```

    Dim swInfrMgr As SldWorks.InterferenceDetectionMgr
    Set swInfrMgr = swAssem.InterferenceDetectionManager

```

```

    swInfrMgr.UseTransform = True

```

```

    Dim swComps(0 To 2) As SldWorks.Component2
    Set swComps(0) = swAssem.GetComponentByName("Part1^AssemInterferenceDetection-1")
    Set swComps(1) = swAssem.GetComponentByName("Part1^AssemInterferenceDetection-2")
    Set swComps(2) = swAssem.GetComponentByName("Part1^AssemInterferenceDetection-3")

```

```

    Dim swXfms(0 To 2) As SldWorks.MathTransform
    Set swXfms(0) = swComps(0).Transform2
    Set swXfms(1) = swComps(1).Transform2
    Set swXfms(2) = swComps(2).Transform2

```

```

' Replacement transform for Part1^AssemInterferenceDetection-2
Dim dXfm(0 To 15) As Double
' Use existing rotation
Dim vXfmCurrent As Variant
vXfmCurrent = swXfms(1).ArrayData

```

```

dXfm(0) = vXfmCurrent(0): dXfm(1) = vXfmCurrent(1): dXfm(2) = vXfmCurrent(2)
dXfm(3) = vXfmCurrent(3): dXfm(4) = vXfmCurrent(4): dXfm(5) = vXfmCurrent(5)
dXfm(6) = vXfmCurrent(6): dXfm(7) = vXfmCurrent(7): dXfm(8) = vXfmCurrent(8)
' Translation from assembly origin
dXfm(9) = 0.9: dXfm(10) = 0.1: dXfm(11) = 0
dXfm(12) = 1
dXfm(13) = 0: dXfm(14) = 0: dXfm(15) = 0: dXfm(1) = 0: dXfm(2) = 0

```

```

Dim swMath As SldWorks.MathUtility
Set swMath = swApp.GetMathUtility

Set swXfms(1) = swMath.CreateTransform(dXfm)

Dim lResult As Long
lResult = swInfrMgr.SetComponentsAndTransforms(swComps, swXfms)
Debug.Print "Transforms applied:"
If lResult = swSetComponentsAndTransformsStatus_e.swSetComponentsAndTransforms_Succeeded Then
    Debug.Print " True"
Else
    Debug.Print " False: " & lResult & " vs. " & swSetComponentsAndTransformsStatus_e.swSetComponentsAndTransforms_Succeeded
    bResult = False
End If

Dim actualNumberOfInterferences As Long
actualNumberOfInterferences = swInfrMgr.GetInterferenceCount
Debug.Print "Number of interferences: "
Debug.Print " " & actualNumberOfInterferences

Stop
' Examine the graphics area to verify
' the interferences, then press F5

Dim bCorrectNumberOfInterferences As Boolean
bCorrectNumberOfInterferences = expectedNumberOfInterferences = actualNumberOfInterferences
Debug.Print "Correct number of interferences:"
If bCorrectNumberOfInterferences Then
    Debug.Print " True"
Else
    Debug.Print " False: " & actualNumberOfInterferences & " vs. " & expectedNumberOfInterferences
    bResult = False
End If

Dim bCorrectInterferences As Boolean
bCorrectInterferences = VerifyInterferencesAgainstExpectation(swInfrMgr.GetInterferences, expectedInterferenceVolumes,
expectedNumberOfInterferenceComps)
Debug.Print "Correct interferences:"
If bCorrectInterferences Then
    Debug.Print " True"
Else
    Debug.Print " False"
    bResult = False
End If

swInfrMgr.Done

Dim bBackToStartingPositions As Boolean
bBackToStartingPositions = VerifyFinishingPositions
Debug.Print "Components back at starting positions:"
If bBackToStartingPositions Then
    Debug.Print " True"
Else
    Debug.Print " False"
    bResult = False
End If

End Sub

Public Function OpenAssembly() As SldWorks.AssemblyDoc
    If swApp Is Nothing Then
        Set swApp = Application.SldWorks
    End If

    swApp.CloseAllDocuments True

    Dim docspec As SldWorks.DocumentSpecification
    Dim assemblyFile As String
    assemblyFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\AssemInterferenceDetection.sldasm"
    Set docspec = swApp.GetOpenDocSpec(assemblyFile)

    Dim swModel As SldWorks.ModelDoc2
    Set swModel = swApp.ActiveDoc
    Set swModel = swApp.OpenDoc7(docspec)

    Set OpenAssembly = swModel
End Function

Public Function EqualWithinTolerance(ByVal A As Double, ByVal B As Double) As Boolean
    Const tolerance As Double = 0.00001
    Dim difference As Double
    difference = Abs(A - B)
    EqualWithinTolerance = difference < tolerance
End Function

Public Sub RecordStartingPositions(swAssem As AssemblyDoc)
    Set colStartingPositions = New Collection

    Dim vComponents As Variant
    vComponents = swAssem.GetComponents(True)

    If Not IsEmpty(vComponents) Then
        Dim vComp As Variant
        For Each vComp In vComponents
            Dim swComp As Component2
            Set swComp = vComp
            Dim position As Class1
            Set position = New Class1
            position.RecordReferencePosition swComp

            colStartingPositions.Add position
        Next
    End If
End Sub

```

```

Public Function VerifyFinishingPositions() As Boolean
    Dim bResult As Boolean
    bResult = True

    Dim position As Class1
    Debug.Print "Position of: "
    For Each position In colStartingPositions
        If position.IsAtReferencePosition() Then
            Debug.Print " " & position.Name & " has not moved"
        Else
            Debug.Print " " & position.Name & " has moved"
            bResult = False
        End If
    Next

    VerifyFinishingPositions = bResult
End Function

Public Function VerifyInterferencesAgainstExpectation(vInterferences As Variant, expectedVolumes As Collection, expectedCompCounts As Collection) As Boolean
    Dim bResult As Boolean
    bResult = True
    If IsEmpty(vInterferences) Then
        Debug.Print "IInterferenceDetectionMgr::GetInterferences returned:"
        If expectedVolumes.Count = 0 Then
            Debug.Print " Nothing as expected"
        Else
            Debug.Print " Nothing unexpectedly"
            bResult = False
        End If
    Else
        Dim numInterferences As Long
        numInterferences = UBound(vInterferences) - LBound(vInterferences) + 1
        If numInterferences <> expectedVolumes.Count Then
            Debug.Print "IInterferenceDetectionMgr::GetInterferences returned unexpected number of interferences: " & numInterferences & " vs. " & expectedVolumes.Count
            bResult = False
        End If

        ' Cannot assume the same order
        ' Try to find matches for each returned interference
        Dim i As Long

        Debug.Print "Match found for calculated interference:"
        For i = LBound(vInterferences) To UBound(vInterferences)
            Dim swInterference As Interference
            Set swInterference = vInterferences(i)

            Dim volume As Double
            volume = swInterference.volume

            Dim compCount As Long
            compCount = swInterference.GetComponentCount

            Dim bFoundMatch As Boolean
            bFoundMatch = False

            Dim j As Long
            For j = 1 To expectedVolumes.Count
                If EqualWithinTolerance(expectedVolumes(j), volume) And (expectedCompCounts(j) = compCount) Then
                    bFoundMatch = True
                    expectedVolumes.Remove j
                    expectedCompCounts.Remove j
                    Exit For
                End If
            Next

            If bFoundMatch Then
                Debug.Print " " & i & ": Volume = " & Format(volume, "0.000000") & " & Component count = " & compCount
            Else
                Debug.Print " " & i & ": Volume = " & Format(volume, "0.000000") & " & Component count = " & compCount
                bResult = False
            End If
        Next

        Dim k As Long
        If expectedVolumes.Count > 0 Then
            For k = 1 To expectedVolumes.Count
                Debug.Print "No match found for expected interference: Volume = " & Format(expectedVolumes(k), "0.000000") & " & Component count = " & expectedCompCounts(k)
            Next
        End If
    End If

    VerifyInterferencesAgainstExpectation = bResult
End Function

Public Sub SelectAllComponents(swAssem As AssemblyDoc)
    Dim swModel As ModelDoc2
    Set swModel = swAssem
    swModel.ClearSelection2 True

    Dim vComponents As Variant
    vComponents = swAssem.GetComponents(True)

    If Not IsEmpty(vComponents) Then
        Dim vComp As Variant
        For Each vComp In vComponents
            Dim swComp As Component2
            Set swComp = vComp
            swComp.Select4 True, Nothing, False
        Next
    End If
End Sub

```

[Back to top](#)

```
'Class1

Option Explicit

Private swComp As Component2
Private swReferenceTransform As MathTransform

Public Sub RecordReferencePosition(swComponent As Component2)
    Set swComp = swComponent
    Set swReferenceTransform = swComp.Transform2
End Sub

Public Function IsAtReferencePosition() As Boolean
    Dim swCurrentTransform As MathTransform
    Set swCurrentTransform = swComp.Transform2

    Dim swProduct As MathTransform
    Set swProduct = swReferenceTransform.Multiply(swCurrentTransform.Inverse)

    Dim vProduct As Variant
    vProduct = swProduct.ArrayData

    ' Require identity transform
    Dim bResult As Boolean
    bResult = EqualWithinTolerance(vProduct(0), 1)
    bResult = EqualWithinTolerance(vProduct(1), 0) And bResult
    bResult = EqualWithinTolerance(vProduct(2), 0) And bResult

    bResult = EqualWithinTolerance(vProduct(3), 0) And bResult
    bResult = EqualWithinTolerance(vProduct(4), 1) And bResult
    bResult = EqualWithinTolerance(vProduct(5), 0) And bResult

    bResult = EqualWithinTolerance(vProduct(6), 0) And bResult
    bResult = EqualWithinTolerance(vProduct(7), 0) And bResult
    bResult = EqualWithinTolerance(vProduct(8), 1) And bResult

    bResult = EqualWithinTolerance(vProduct(9), 0) And bResult
    bResult = EqualWithinTolerance(vProduct(10), 0) And bResult
    bResult = EqualWithinTolerance(vProduct(11), 0) And bResult

    bResult = EqualWithinTolerance(vProduct(12), 1) And bResult

    If Not bResult Then
        Debug.Print Name & ":"
        Debug.Print Format(vProduct(0), "0.000000") & vbTab & Format(vProduct(1), "0.000000") & vbTab & Format(vProduct(2), "0.000000")
        Debug.Print Format(vProduct(3), "0.000000") & vbTab & Format(vProduct(4), "0.000000") & vbTab & Format(vProduct(5), "0.000000")
        Debug.Print Format(vProduct(6), "0.000000") & vbTab & Format(vProduct(7), "0.000000") & vbTab & Format(vProduct(8), "0.000000")
        Debug.Print Format(vProduct(9), "0.000000") & vbTab & Format(vProduct(10), "0.000000") & vbTab & Format(vProduct(11), "0.000000")
        Debug.Print Format(vProduct(12), "0.000000")
    End If

    IsAtReferencePosition = bResult
End Function

Public Property Get Name() As String
    Name = swComp.Name2
End Property
```

[Back to top](#)

Set Components and Transforms for Interference Detection Example (VB.NET)

This example shows how to:

- move an assembly component to an interfering position by replacing its transform.
- detect interference.

This example also shows how to apply absolute transforms to all components in order to multiply by identity transforms, which results in the components remaining in their current positions during interference detection.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Copy and paste Main into SolidWorksMacro.vb of your macro.
' 3. Click Project > Add Class and copy and paste Class1 into Class1.vb.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly.
' 2. Gets the components of the assembly and their transforms.
' 3. Creates a new transform for Part1^AssemInterferenceDetection-2.
' 4. Sets the components and their transforms, using the existing
'    rotation and the translation from the assembly origin.
'    a. Gets the number of interferences.
'    b. Examine the graphics area at the Stop statement to visually
'       verify the interferences, then press F5.
'    c. Programmatically verifies that the number of interferences
'       is as expected.
'    d. Gets whether:
'       * each returned interference is correct.
'       * the components remained in their starting positions during
'         interference detection.
'       * the components are at their starting positions.
' 5. Examine the Immediate window and graphics area.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
'Main
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim colStartingPositions As Collection

    Public Sub main()

        Dim Class1 As New Class1
        Class1 = New Class1

        Dim expectedNumberOfInterferences As Integer
        expectedNumberOfInterferences = 2

        Dim expectedInterferenceVolumes As Collection
        expectedInterferenceVolumes = New Collection

        expectedInterferenceVolumes.Add(0.09)
        expectedInterferenceVolumes.Add(0.01)

        Dim expectedNumberOfInterferenceComps As Collection
        expectedNumberOfInterferenceComps = New Collection

        expectedNumberOfInterferenceComps.Add(2)
        expectedNumberOfInterferenceComps.Add(2)

        Dim bResult As Boolean
        bResult = True

        Dim swAssem As AssemblyDoc
        swAssem = OpenAssembly()

        RecordStartingPositions(swAssem)
        SelectAllComponents(swAssem)
        Dim swInfrMgr As InterferenceDetectionMgr
        swInfrMgr = swAssem.InterferenceDetectionManager

        swInfrMgr.UseTransform = True

        Dim swComps(0 To 2) As Component2
        swComps(0) = swAssem.GetComponentByName("Part1^AssemInterferenceDetection-1")
        swComps(1) = swAssem.GetComponentByName("Part1^AssemInterferenceDetection-2")
        swComps(2) = swAssem.GetComponentByName("Part1^AssemInterferenceDetection-3")
        Dim swXfms(0 To 2) As MathTransform
        swXfms(0) = swComps(0).Transform2
        swXfms(1) = swComps(1).Transform2
        swXfms(2) = swComps(2).Transform2

        ' Replacement transform for Part1^AssemInterferenceDetection-2
        Dim dXfm(0 To 15) As Double

        ' Use existing rotation
        Dim vXfmCurrent As Object

        vXfmCurrent = swXfms(1).ArrayData
        dXfm(0) = vXfmCurrent(0) : dXfm(1) = vXfmCurrent(1) : dXfm(2) = vXfmCurrent(2)
        dXfm(3) = vXfmCurrent(3) : dXfm(4) = vXfmCurrent(4) : dXfm(5) = vXfmCurrent(5)
        dXfm(6) = vXfmCurrent(6) : dXfm(7) = vXfmCurrent(7) : dXfm(8) = vXfmCurrent(8)

        ' Translation from assembly origin
        dXfm(9) = 0.9 : dXfm(10) = 0.1 : dXfm(11) = 0
        dXfm(12) = 1

```

```

dxfm(13) = 0 : dxfm(14) = 0 : dxfm(15) = 0 : dxfm(1) = 0 : dxfm(2) = 0

Dim swMath As MathUtility
swMath = swApp.GetMathUtility
swXfms(1) = swMath.CreateTransform(dxfm)

Dim lResult As Integer
lResult = swInfrMgr.SetComponentsAndTransforms(swComps, swXfms)
Debug.Print("Transforms applied:")
If lResult = swSetComponentsAndTransformsStatus_e.swSetComponentsAndTransforms_Succeeded Then
    Debug.Print(" True")
Else
    Debug.Print(" False: " & lResult & " vs. " & swSetComponentsAndTransformsStatus_e.swSetComponentsAndTransforms_Succeeded)
    bResult = False
End If

Dim actualNumberOfInterferences As Integer
actualNumberOfInterferences = swInfrMgr.GetInterferenceCount
Debug.Print("Number of interferences: ")
Debug.Print(" " & actualNumberOfInterferences)

Stop
' Examine the graphics area to verify
' the interferences, then press F5

Dim bCorrectNumberOfInterferences As Boolean
bCorrectNumberOfInterferences = expectedNumberOfInterferences = actualNumberOfInterferences
Debug.Print("Correct number of interferences:")
If bCorrectNumberOfInterferences Then
    Debug.Print(" True")
Else
    Debug.Print(" False: " & actualNumberOfInterferences & " vs. " & expectedNumberOfInterferences)
    bResult = False
End If

Dim bCorrectInterferences As Boolean
bCorrectInterferences = VerifyInterferencesAgainstExpectation(swInfrMgr.GetInterferences, expectedInterferenceVolumes, expectedNumberOfInterferenc
eComps)
Debug.Print("Correct interferences:")
If bCorrectInterferences Then
    Debug.Print(" True")
Else
    Debug.Print(" False")
    bResult = False
End If

swInfrMgr.Done()

Dim bBackToStartingPositions As Boolean
bBackToStartingPositions = VerifyFinishingPositions()
Debug.Print("Components back at starting positions:")
If bBackToStartingPositions Then
    Debug.Print(" True")
Else
    Debug.Print(" False")
    bResult = False
End If

End Sub

Public Function OpenAssembly() As AssemblyDoc
    swApp.CloseAllDocuments(True)
    Dim docspec As DocumentSpecification
    Dim assemblyFile As String
    assemblyFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\AssemInterferenceDetection.sldasm"
    docspec = swApp.GetOpenDocSpec(assemblyFile)
    Dim swModel As ModelDoc2
    swModel = swApp.ActiveDoc
    swModel = swApp.OpenDoc7(docspec)
    OpenAssembly = swModel
End Function

Public Shared Function EqualWithinTolerance(ByVal A As Double, ByVal B As Double) As Boolean
    Const tolerance As Double = 0.00001
    Dim difference As Double
    difference = Math.Abs(A - B)
    EqualWithinTolerance = difference < tolerance
End Function

Public Sub RecordStartingPositions(ByVal swAssem As AssemblyDoc)
    colStartingPositions = New Collection
    Dim vComponents As Object
    vComponents = swAssem.GetComponents(True)
    If Not IsNothing(vComponents) Then
        Dim vComp As Object
        For Each vComp In vComponents
            Dim swComp As Component2
            swComp = vComp
            Dim position As Class1
            position = New Class1
            position.RecordReferencePosition(swComp)
            colStartingPositions.Add(position)
        Next
    End If
End Sub

Public Function VerifyFinishingPositions() As Boolean
    Dim bResult As Boolean
    bResult = True
    Dim position As Class1
    Debug.Print("Position of: ")
    For Each position In colStartingPositions
        If position.IsAtReferencePosition() Then
            Debug.Print(" " & position.Name & " has not moved")
        Else
            Debug.Print(" " & position.Name & " has moved")
            bResult = False
        End If
    Next
    VerifyFinishingPositions = bResult

```

```

End Function

Public Function VerifyInterferencesAgainstExpectation(ByVal vInterferences As Object, ByVal expectedVolumes As Collection, ByVal expectedCompCounts As
Collection) As Boolean
    Dim bResult As Boolean
    bResult = True
    If IsNothing(vInterferences) Then
        Debug.Print("IInterferenceDetectionMgr::GetInterferences returned:")
        If expectedVolumes.Count = 0 Then
            Debug.Print(" Nothing as expected")
        Else
            Debug.Print(" Nothing unexpectedly")
            bResult = False
        End If
    Else
        Dim numInterferences As Integer
        numInterferences = UBound(vInterferences) - LBound(vInterferences) + 1
        If numInterferences <> expectedVolumes.Count Then
            Debug.Print("IInterferenceDetectionMgr::GetInterferences returned unexpected number of interferences: " & numInterferences & " vs. " & exp
ectedVolumes.Count)
            bResult = False
        End If
        ' Cannot assume the same order
        ' Try to find matches for each returned interference
        Dim i As Integer
        Debug.Print("Match found for calculated interference:")
        For i = LBound(vInterferences) To UBound(vInterferences)
            Dim swInterference As Interference
            swInterference = vInterferences(i)
            Dim volume As Double
            volume = swInterference.Volume
            Dim compCount As Integer
            compCount = swInterference.GetComponentCount
            Dim bFoundMatch As Boolean
            bFoundMatch = False
            Dim j As Integer
            For j = 1 To expectedVolumes.Count
                If EqualWithinTolerance(expectedVolumes(j), volume) And (expectedCompCounts(j) = compCount) Then
                    bFoundMatch = True
                    expectedVolumes.Remove(j)
                    expectedCompCounts.Remove(j)
                    Exit For
                End If
            Next
            If bFoundMatch Then
                Debug.Print(" " & i & ": Volume = " & Format(volume, "0.000000") & " & Component count = " & compCount)
            Else
                Debug.Print(" " & i & ": Volume = " & Format(volume, "0.000000") & " & Component count = " & compCount)
                bResult = False
            End If
        Next
        Dim k As Integer
        If expectedVolumes.Count > 0 Then
            For k = 1 To expectedVolumes.Count
                Debug.Print("No match found for expected interference: Volume = " & Format(expectedVolumes(k), "0.000000") & " & Component count = " &
expectedCompCounts(k))
            Next
        End If
    End If
    VerifyInterferencesAgainstExpectation = bResult
End Function

Public Sub SelectAllComponents(ByVal swAssem As AssemblyDoc)
    Dim swModel As ModelDoc2
    swModel = swAssem
    swModel.ClearSelection2(True)
    Dim vComponents As Object
    vComponents = swAssem.GetComponents(True)
    If Not IsNothing(vComponents) Then
        Dim vComp As Object
        For Each vComp In vComponents
            Dim swComp As Component2
            swComp = vComp
            swComp.Select4(True, Nothing, False)
        Next
    End If
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

[Back to top](#)

'Class

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports System.Runtime.InteropServices
Imports Macro1VBNET.vbproj.SolidWorksMacro

```

Public Class Class1

```

    Private swComp As Component2
    Private swReferenceTransform As MathTransform

```

```

    Public Sub RecordReferencePosition(ByVal swComponent As Component2)
        swComp = swComponent
        swReferenceTransform = swComp.Transform2
    End Sub

```

```

    Public Function IsAtReferencePosition() As Boolean
        Dim swCurrentTransform As MathTransform
        swCurrentTransform = swComp.Transform2
        Dim swProduct As MathTransform
        swProduct = swReferenceTransform.Multiply(swCurrentTransform.Inverse)
    End Function

```

```

Dim vProduct As Object
vProduct = swProduct.ArrayData
' Require identity transform
Dim bResult As Boolean
bResult = EqualWithinTolerance(vProduct(0), 1)
bResult = EqualWithinTolerance(vProduct(1), 0) And bResult
bResult = EqualWithinTolerance(vProduct(2), 0) And bResult

bResult = EqualWithinTolerance(vProduct(3), 0) And bResult
bResult = EqualWithinTolerance(vProduct(4), 1) And bResult
bResult = EqualWithinTolerance(vProduct(5), 0) And bResult

bResult = EqualWithinTolerance(vProduct(6), 0) And bResult
bResult = EqualWithinTolerance(vProduct(7), 0) And bResult
bResult = EqualWithinTolerance(vProduct(8), 1) And bResult

bResult = EqualWithinTolerance(vProduct(9), 0) And bResult
bResult = EqualWithinTolerance(vProduct(10), 0) And bResult
bResult = EqualWithinTolerance(vProduct(11), 0) And bResult

bResult = EqualWithinTolerance(vProduct(12), 1) And bResult

If Not bResult Then
    Debug.Print(Name & ":")
    Debug.Print(Format(vProduct(0), "0.000000") & vbTab & Format(vProduct(1), "0.000000") & vbTab & Format(vProduct(2), "0.000000"))
    Debug.Print(Format(vProduct(3), "0.000000") & vbTab & Format(vProduct(4), "0.000000") & vbTab & Format(vProduct(5), "0.000000"))
    Debug.Print(Format(vProduct(6), "0.000000") & vbTab & Format(vProduct(7), "0.000000") & vbTab & Format(vProduct(8), "0.000000"))
    Debug.Print(Format(vProduct(9), "0.000000") & vbTab & Format(vProduct(10), "0.000000") & vbTab & Format(vProduct(11), "0.000000"))
    Debug.Print(Format(vProduct(12), "0.000000"))
End If

IsAtReferencePosition = bResult
End Function

Public ReadOnly Property Name() As String
    Get
        Name = swComp.Name2
    End Get
End Property

End Class

```

[Back to top](#)

Set Components Transparent Example (VBA)

This example shows how to make one or more selected components transparent

```
'-----  
'  
' Preconditions: Assembly document is open and one  
'           or more components are selected.  
'  
' Postconditions: Selected components are transparent.  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModelDoc As SldWorks.ModelDoc2  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim boolstatus As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModelDoc = swApp.ActiveDoc  
    Set swAssembly = swModelDoc  
  
    ' Set the selected component to transparent  
    boolstatus = swAssembly.SetComponentTransparent(True)  
  
    'Set the selected component to not transparent  
    'boolstatus = swAssembly.SetComponentTransparent(False)  
  
End Sub
```

Set Custom Bend Deduction Example (C#)

This example shows how to set a custom bend deduction for a miter flange feature.

```
//-----
// Preconditions:
// 1. Open a sheet metal part that contains a miter flange
//    feature with bends.
// 2. Select the miter flange feature.
//
// Postconditions: Sets the selected miter flange's custom
// bend deduction to 1 mm.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CustomBendAllowanceType_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        bool boolstatus;
        MiterFlangeFeatureData FeatureData;
        Feature Feature;
        CustomBendAllowance bData;
        SelectionMgr selMgr;
        Component component = null;

        string nam;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            // Get the selected feature
            selMgr = (SelectionMgr)Part.SelectionManager;
            Feature = (Feature)selMgr.GetSelectedObject6(1, -1);

            // Get the name of the selected feature
            nam = Feature.GetTypeName();

            // Get the feature definition
            FeatureData = (MiterFlangeFeatureData)Feature.GetDefinition();

            // Get whether to use default bend allowance to determine the current state
            bool useCustom = false;
            useCustom = FeatureData.UseDefaultBendAllowance;

            // Get the custom bend allowance object
            bData = FeatureData.GetCustomBendAllowance();

            long bType = 0;
            bType = bData.Type;
            // Set the bend allowance type to deduction
            bData.Type = (int)swBendAllowanceTypes_e.swBendAllowanceDeduction;

            // Set the value of the bend deduction
            bData.BendDeduction = 0.001;

            FeatureData.UseDefaultBendAllowance = false;

            // Set the value of the custom bend deduction
            FeatureData.SetCustomBendAllowance(bData);

            // Modify the feature definition
            boolstatus = Feature.ModifyDefinition(FeatureData, Part, component);
        }

        public SldWorks swApp;
    }
}
```

Set Custom Bend Deduction Example (VBA)

This example shows how to set a custom bend deduction for a miter flange feature.

```
'-----
' Preconditions:
' 1. Open a sheet metal part that contains a miter flange
'    feature with bends.
' 2. Select the miter flange feature.
'
' Postconditions: Sets the selected miter flange's custom
' bend deduction to 1 mm.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim boolstatus As Boolean
Dim FeatureData As SldWorks.MiterFlangeFeatureData
Dim Feature As SldWorks.Feature
Dim Component As SldWorks.Component
Dim bData As SldWorks.CustomBendAllowance
Dim nam As String

Sub main()
    Set swApp = CreateObject("SldWorks.Application")
    Set Part = swApp.ActiveDoc

    ' Get the selected feature
    Set Feature = Part.SelectionManager.GetSelectedObject6(1, -1)

    ' Get the name of the selected feature
    nam = Feature.GetTypeName

    ' Get the feature definition
    Set FeatureData = Feature.GetDefinition

    ' Get whether to use default bend allowance to determine the current state
    Dim useCustom As Boolean
    useCustom = FeatureData.UseDefaultBendAllowance

    ' Get the custom bend allowance object
    Set bData = FeatureData.GetCustomBendAllowance

    Dim bType As Long
    bType = bData.Type
    ' Set the bend allowance type to deduction
    bData.Type = swBendAllowanceDeduction

    ' Set the value of the bend deduction
    bData.BendDeduction = 0.001

    FeatureData.UseDefaultBendAllowance = False
    ' Set the value of the custom bend deduction
    Call FeatureData.SetCustomBendAllowance(bData)

    ' Modify the feature definition
    boolstatus = Feature.ModifyDefinition(FeatureData, Part, Component)
End Sub
```

Set Custom Bend Deduction Example (VB.NET)

This example shows how to set a custom bend deduction for a miter flange feature.

```

'-----
' Preconditions:
' 1. Open a sheet metal part that contains a miter flange
'   feature with bends.
' 2. Select the miter flange feature.
'
' Postconditions: Sets the selected miter flange's custom
' bend deduction to 1 mm.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```

Partial Class SolidWorksMacro

```

```

    Dim Part As ModelDoc2
    Dim boolstatus As Boolean
    Dim FeatureData As MiterFlangeFeatureData
    Dim Feature As Feature
    Dim Component As Component
    Dim bData As CustomBendAllowance
    Dim nam As String

    Sub main()

        Part = swApp.ActiveDoc

        ' Get the selected feature
        Feature = Part.SelectionManager.GetSelectedObject6(1, -1)

        ' Get the name of the selected feature
        nam = Feature.GetTypeName

        ' Get the feature definition
        FeatureData = Feature.GetDefinition

        ' Get whether to use default bend allowance to determine the current state
        Dim useCustom As Boolean
        useCustom = FeatureData.UseDefaultBendAllowance

        ' Get the custom bend allowance object
        bData = FeatureData.GetCustomBendAllowance

        Dim bType As Long
        bType = bData.Type
        ' Set the bend allowance type to deduction
        bData.Type = swBendAllowanceTypes_e.swBendAllowanceDeduction

        ' Set the value of the bend deduction
        bData.BendDeduction = 0.001

        FeatureData.UseDefaultBendAllowance = False

        ' Set the value of the custom bend deduction
        Call FeatureData.SetCustomBendAllowance(bData)

        ' Modify the feature definition
        boolstatus = Feature.ModifyDefinition(FeatureData, Part, Component)
    End Sub

```

```

End Sub

```

```

Public swApp As SldWorks

```

```

End Class

```

Set Dimensions to Mid-Tolerance Example (VBA)

This example:

- shows how to iterate over all dimensions in a part and set each dimension with tolerance to the middle of the tolerance range.
- does not analyze the overall effect of varying tolerances.

```

'-----
' Preconditions:
' 1. Open a part that has tolerances set.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a derived configuration based on the current
'    configuration.
' 2. Makes the derived configuration the active configuration.
' 3. Sets all dimensions in the derived configuration with tolerances
'    to mid-tolerance.
' 4. Removes all tolerances from the dimensions.
' 5. Examine the Immediate window.
'-----

Option Explicit

Function GetDimFactor(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swDim As SldWorks.Dimension) As Double
    Const PI As Double = 3.14159265
    Const LEN_FACTOR As Double = 1000#
    Const ANG_FACTOR As Double = 180# / PI

    Select Case swDim.GetType
        Case swDimensionParamTypeDoubleLinear
            GetDimFactor = LEN_FACTOR
        Case swDimensionParamTypeDoubleAngular
            GetDimFactor = ANG_FACTOR
        Case Else
            Debug.Assert False
    End Select
End Function

Function GetDimString(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swDim As SldWorks.Dimension) As String
    Const LEN_STR As String = " mm"
    Const ANG_STR As String = " deg"

    Select Case swDim.GetType
        Case swDimensionParamTypeDoubleLinear
            GetDimString = LEN_STR
        Case swDimensionParamTypeDoubleAngular
            GetDimString = ANG_STR
        Case Else
            Debug.Assert False
    End Select
End Function

Sub SetDimensionToMidTolerance(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swDim As SldWorks.Dimension)
    Dim nRetVal As Long
    Dim nDimFactor As Double
    Dim sDimStr As String
    Dim vTolVal As Variant
    Dim nOldVal As Double
    Dim nNewVal As Double
    Dim bRet As Boolean
    If swDimensionDriving <> swDim.DrivenState Or swDim.ReadOnly Then
        Exit Sub
    End If

    ' Might have tolerance values stored from
    ' previous tolerance type, so check current
    ' tolerance type first
    If swTolNONE = swDim.GetToleranceType Then
        Exit Sub
    End If

    ' Mid-tolerance cannot make sense for a MIN/MAX or symmetric tolerance
    If swTolMIN = swDim.GetToleranceType Or swTolMAX = swDim.GetToleranceType Or swTolSYMMETRIC = swDim.GetToleranceType Then
        Exit Sub
    End If

    vTolVal = swDim.GetToleranceValues
    If IsEmpty(vTolVal) Then
        Exit Sub
    End If

    nDimFactor = GetDimFactor(swApp, swModel, swDim)
    sDimStr = GetDimString(swApp, swModel, swDim)

    nOldVal = swDim.GetSystemValue2("")
    nNewVal = nOldVal + (vTolVal(0) + vTolVal(1)) / 2#
    Debug.Print "    -->"
    Debug.Print "        Old Value    = " & nOldVal * nDimFactor & sDimStr
    Debug.Print "        New Value    = " & nNewVal * nDimFactor & sDimStr
    nRetVal = swDim.SetSystemValue3(nNewVal, swSetValue_InThisConfiguration, Empty): Debug.Assert swSetValue_Successful = nRetVal

    ' Changed to mid tolerance, so remove tolerance;
    ' otherwise, change dimension when run again
    bRet = swDim.SetToleranceType(swTolNONE): Debug.Assert bRet
End Sub

```

```

Sub ProcessDimension(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swDim As SldWorks.Dimension)
    Dim nDimFactor As Double
    Dim sDimStr As String
    Dim vTolVal As Variant
    Dim bRet As Boolean

    nDimFactor = GetDimFactor(swApp, swModel, swDim)
    sDimStr = GetDimString(swApp, swModel, swDim)

    Debug.Print " " & swDim.FullName
    Debug.Print " Value = " & swDim.GetSystemValue2("") * nDimFactor & sDimStr
    Debug.Print " Driven = " & swDim.DrivenState
    Debug.Print " ReadOnly = " & swDim.ReadOnly

    vTolVal = swDim.GetToleranceValues
    If IsEmpty(vTolVal) Then
        Debug.Print " No tolerance information"
        Exit Sub
    End If

    Debug.Print " TolType = " & swDim.GetToleranceType
    Debug.Print " FitType = " & swDim.GetToleranceFitValues
    Debug.Print " MaxTol = " & vTolVal(1) * nDimFactor & sDimStr
    Debug.Print " MinTol = " & vTolVal(0) * nDimFactor & sDimStr
End Sub

Sub ProcessMassProps(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2)
    Dim swDocExt As SldWorks.ModelDocExtension
    Dim swMass As SldWorks.MassProperty

    Set swDocExt = swModel.Extension
    Set swMass = swDocExt.CreateMassProperty
    Debug.Print " Mass = " & swMass.Mass * 1000# & " g"
    Debug.Print " Surface Area = " & swMass.SurfaceArea * 1000000# & " mm^2"
    Debug.Print " Volume = " & swMass.Volume * 1000000000# & " mm^3"
    Debug.Print " Density = " & swMass.Density & " kg/m^3"
    Debug.Print " CenterOfMass = (" & swMass.CenterOfMass(0) * 1000# & ", " & swMass.CenterOfMass(1) * 1000# & ", " & swMass.CenterOfMass(2) * 1000# & ") mm"
End Sub

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swConf As SldWorks.Configuration
    Dim swMidConf As SldWorks.Configuration
    Dim swConfMgr As SldWorks.ConfigurationManager
    Dim swFeat As SldWorks.Feature
    Dim swDispDim As SldWorks.DisplayDimension
    Dim swDim As SldWorks.Dimension
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swConf = swModel.GetActiveConfiguration
    Set swConfMgr = swModel.ConfigurationManager
    Set swMidConf = swConfMgr.AddConfiguration(swConf.Name & " - mid tolerance", "mid tolerance", "mid tolerance", 1, swConf.Name, "mid tolerance"):
    Debug.Assert Not swMidConf Is Nothing
    Set swFeat = swModel.FirstFeature
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Nominal Tolerance:"
    ProcessMassProps swApp, swModel
    Debug.Print " -----"

    Do While Not swFeat Is Nothing
        Debug.Print " " & swFeat.Name
        Set swDispDim = swFeat.GetFirstDisplayDimension
        Do While Not swDispDim Is Nothing
            Set swDim = swDispDim.GetDimension
            ProcessDimension swApp, swModel, swDim
            SetDimensionToMidTolerance swApp, swModel, swDim
            Set swDispDim = swFeat.GetNextDisplayDimension(swDispDim)
        Loop
        Set swFeat = swFeat.GetNextFeature
    Loop

    bRet = swModel.ForceRebuild3(False): Debug.Assert bRet

    Debug.Print " Middle Tolerance:"
    ProcessMassProps swApp, swModel
    Debug.Print " -----"
End Sub

```

Set Display Mode of View Example (C#)

This example shows how to set the display mode of a drawing view.

```
// -----  
// Preconditions:  
// 1. Open a drawing and select a drawing view.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Gets the selected view's current display mode.  
// 2. Resets the display mode.  
// 3. Gets the new display mode.  
// 4. Examine the Immediate window and graphics area.  
// -----  
  
using System;  
using System.Collections.Generic;  
using System.Diagnostics;  
using System.Globalization;  
using System.IO;  
using System.Linq;  
using System.Reflection;  
using System.Runtime.CompilerServices;  
using System.Security;  
using System.Text;  
using System.Threading.Tasks;  
using Microsoft.VisualBasic;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
  
namespace SetDisplayMode_CSharp  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            DrawingDoc swDraw;  
            Sheet swSheet;  
            View swView;  
            SelectionMgr swSelectionMgr;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swDraw = (DrawingDoc)swModel;  
            swSheet = (Sheet)swDraw.GetCurrentSheet();  
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;  
  
            swView = (View)swSelectionMgr.GetSelectedObject6(1, -1);  
  
            Debug.Print("====Current Display Mode====");  
            Debug.Print("");  
  
            bool UseParentProp;  
            UseParentProp = swView.GetUseParentDisplayMode();  
            Debug.Print("Using parent view mode? " + UseParentProp);  
  
            long displayMode;  
            displayMode = swView.GetDisplayMode2();  
        }  
    }  
}
```

```

    Debug.Print("Current display mode as defined by swDisplayMode_e: " + displayMode);

    bool Facetted;
    Facetted = swView.GetFacettedHlrDisplay();
    Debug.Print("Faceted (draft quality)? " + Facetted);

    bool EdgesMode;
    EdgesMode = swView.GetDisplayEdgesInShadedMode();
    Debug.Print("Display edges when the view is in shaded mode? " + EdgesMode);

    bool cThreadQuality;
    cThreadQuality = swView.GetCThreadQuality();
    Debug.Print("Precision cosmetic thread display quality? " + cThreadQuality);

    // Reset display mode: Do not use parent view's display mode, shaded mode, draft quality, no edges displayed,
    precision cosmetic threads
    swView.SetDisplayMode4(false, 3, true, false, true);

    Debug.Print("");
    Debug.Print("====After Setting Display Mode====");
    Debug.Print("");
    Debug.Print("Using parent view mode? " + swView.GetUseParentDisplayMode());
    Debug.Print("Current display mode as defined by swDisplayMode_e: " + swView.GetDisplayMode2());
    Debug.Print("Faceted (draft quality)? " + swView.GetFacettedHlrDisplay());
    Debug.Print("Display edges when view is in shaded mode? " + swView.GetDisplayEdgesInShadedMode());
    Debug.Print("Precision cosmetic thread display quality? " + swView.GetCThreadQuality());
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```

Set Display Mode of View Example (VB.NET)

This example shows how to set the display mode of a drawing view.

```
'-----
' Preconditions:
' 1. Open a drawing and select a drawing view.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the selected view's current display mode.
' 2. Resets the display mode.
' 3. Gets the new display mode.
' 4. Examine the Immediate window and graphics area.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim swDraw As DrawingDoc
        Dim swSheet As Sheet
        Dim swView As View
        Dim swSelectionMgr As SelectionMgr

        swModel = swApp.ActiveDoc
        swDraw = swModel
        swSheet = swDraw.GetCurrentSheet
        swSelectionMgr = swModel.SelectionManager

        swView = swSelectionMgr.GetSelectedObject6(1, -1)

        Debug.Print("====Current Display Mode====")
        Debug.Print("")

        Dim UseParentProp As Boolean
        UseParentProp = swView.GetUseParentDisplayMode
        Debug.Print("Using parent view mode? " & UseParentProp)

        Dim displayMode As Long
        displayMode = swView.GetDisplayMode2
        Debug.Print("Current display mode as defined by swDisplayMode_e: " & displayMode)

        Dim Facetted As Boolean
        Facetted = swView.GetFacettedHlrDisplay
        Debug.Print("Facetted (draft quality)? " & Facetted)

        Dim EdgesMode As Boolean
        EdgesMode = swView.GetDisplayEdgesInShadedMode
        Debug.Print("Display edges when the view is in shaded mode? " & EdgesMode)

        Dim cThreadQuality As Boolean
        cThreadQuality = swView.GetCThreadQuality
        Debug.Print("Precision cosmetic thread display quality? " & cThreadQuality)
```

```

'Reset display mode: Do not use parent view's display mode, shaded mode, draft quality, no edges displayed, precision
cosmetic threads
swView.SetDisplayMode4(False, 3, True, False, True)

Debug.Print("")
Debug.Print("====After Setting Display Mode====")
Debug.Print("")
Debug.Print("Using parent view mode? " & swView.GetUseParentDisplayMode)
Debug.Print("Current display mode as defined by swDisplayMode_e: " & swView.GetDisplayMode2)
Debug.Print("Faceted (draft quality)? " & swView.GetFacettedHlrDisplay)
Debug.Print("Display edges when view is in shaded mode? " & swView.GetDisplayEdgesInShadedMode)
Debug.Print("Precision cosmetic thread display quality? " & swView.GetCThreadQuality)

End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Set Drawing Sheet Properties Example (C#)

This example shows how to set the properties of a drawing sheet.

```
//-----
// Preconditions:
// 1. Create a new drawing document.
// 2. Add another sheet to the drawing.
// 3. Select the Tools > Options > Document Properties >
// Drawing Sheets > Use custom property values from the sheet
// check box if it is not selected.
// 4. Right-click Sheet2, click Properties, clear the Same as sheet
// specified in Document Properties check box if it is selected, and
// click OK to close the Sheet Properties dialog.
//
// Postconditions:
// 1. Selects the Same as sheet specified in Document Properties
// check box on the Sheet Properties dialog.
// 2. Right-click Sheet2 and click Properties to verify step 1.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            DrawingDoc swDrawing = default(DrawingDoc);
            Sheet swSheet = default(Sheet);
            double[] sheetProperties = null;

            swDrawing = (DrawingDoc)swApp.ActiveDoc;

            // Active sheet is Sheet2
            swSheet = (Sheet)swDrawing.GetCurrentSheet();
            sheetProperties = (double[])swSheet.GetProperties2();
            int prop1;
            int prop2;
            bool prop5;
            bool prop8;
            prop1 = (int)System.Convert.ToInt32(sheetProperties[0]);
            prop2 = (int)System.Convert.ToInt32(sheetProperties[1]);
            prop5 = (bool)System.Convert.ToBoolean(sheetProperties[4]);
            prop8 = (bool)System.Convert.ToBoolean(sheetProperties[7]);
            prop8 = true;
            swSheet.SetProperties2(prop1, prop2, sheetProperties[2], sheetProperties[3], prop5, sheetProperties[5], sheetProperties[6], prop8);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Set Drawing Sheet Properties Example (VBA)

This example shows how to set the properties of a drawing sheet.

```
'-----  
' Preconditions:  
' 1. Create a new drawing document.  
' 2. Add another sheet to the drawing.  
' 3. Select the Tools > Options > Document Properties >  
'   Drawing Sheets > Use custom property values from the sheet  
'   check box if it is not selected.  
' 4. Right-click Sheet2, click Properties, clear the Same as sheet  
'   specified in Document Properties check box if it is selected, and  
'   click OK to close the Sheet Properties dialog.  
'  
' Postconditions:  
' 1. Selects the Same as sheet specified in Document Properties  
'   check box on the Sheet Properties dialog.  
' 2. Right-click Sheet2 and click Properties to verify step 1.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swDrawing As SldWorks.DrawingDoc  
Dim swSheet As SldWorks.Sheet  
Dim sheetProperties As Variant
```

Sub main()

```
    Set swApp = Application.SldWorks  
    Set swDrawing = swApp.ActiveDoc  
  
    ' Active sheet is Sheet2  
    Set swSheet = swDrawing.GetCurrentSheet  
    sheetProperties = swSheet.GetProperties2  
    sheetProperties(7) = 1#  
    swSheet.SetProperties2 sheetProperties(0), sheetProperties(1), sheetProperties(2), sheetProperties(3), sheetProperties(4), sheetProperties(5),  
    sheetProperties(6), sheetProperties(7)
```

End Sub

Set Drawing Sheet Properties Example (VB.NET)

This example shows how to set the properties of a drawing sheet.

```
-----
Preconditions:
1. Create a new drawing document.
2. Add another sheet to the drawing.
3. Select the Tools > Options > Document Properties > Drawing Sheets > Use custom property values from the sheet check box if it is not selected.
4. Right-click Sheet2, click Properties, clear the Same as sheet specified in Document Properties check box if it is selected, and click OK to close the Sheet Properties dialog.
-----
Postconditions:
1. Selects the Same as sheet specified in Document Properties check box on the Sheet Properties dialog.
2. Right-click Sheet2 and click Properties to verify step 1.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swDrawing As DrawingDoc
        Dim swSheet As Sheet
        Dim sheetProperties As Object

        swDrawing = swApp.ActiveDoc

        ' Active sheet is Sheet2
        swSheet = swDrawing.GetCurrentSheet
        sheetProperties = swSheet.GetProperties2
        sheetProperties(7) = 1.0#
        swSheet.SetProperties2(sheetProperties(0), sheetProperties(1), sheetProperties(2), sheetProperties(3), sheetProperties(4), sheetProperties(5), sheetProperties(6), sheetProperties(7))

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Set Focus on PropertyManager Page Control Example (C#)

This example shows how to set focus on a PropertyManager page control.

```
//-----
// Preconditions:
// 1. Open a model document.
// 2. Copy Main to the main module.
// 3. Click Project > Add Class > Add and copy Class1 to Class1.cs.
// 4. Click Project > Add Reference > browse to install_dir\api\redist,
// select SolidWorks.Interop.swpublished.dll < OK.
// 5. Open the Immediate window.
//
// Postconditions:
// 1. Creates and displays a PropertyManager page.
// 2. Select the check box to set focus on Text box.
// 3. Examine the Immediate window.
// 4. Clear the check box to remove focus from Text box.
// 5. Examine the Immediate window.
// 6. Click OK to close the PropertyManager page.
//-----
//Main
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public ModelDoc2 Part;
        public Class1 pm;

        public void Main()
        {
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, false);
            Part = (ModelDoc2)swApp.ActiveDoc;

            //Create a new instance of the PropertyManager class
            pm = new Class1(swApp);
            pm.Show();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

[Back to top](#)

```
//Class1
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swpublished;
using System;
using System.Runtime.InteropServices;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    [ComVisibleAttribute(true)]
    public class Class1 : PropertyManagerPage2Handler9
    {
        //General objects required for the PropertyManager page
        PropertyManagerPage2 pm_Page;
        PropertyManagerPageGroup pm_Groupbox;
        PropertyManagerPageCheckbox pm_Checkbox;
        PropertyManagerPageTextbox pm_Textbox;

        //Each object in the page needs a unique ID
        const int groupboxID = 1;
        const int checkboxID = 2;
        const int textboxID = 3;

        public void Show()
        {
            pm_Page.Show2(0);
        }

        //The following runs when a new instance
        //of the class is created
        public Class1(SldWorks swApp)
        {
            string pageTitle = null;
            string caption = null;
            long options = 0;
            int errors = 0;
            int control = 0;
            int alignment = 0;

            //Set the variables for the page
            pageTitle = "Test focus methods";
            options = (int)swPropertyManagerButtonTypes_e.swPropertyManager_OkayButton + (int)swPropertyManagerButtonTypes_e.swPropo
```

```

//Create the PropertyManager page
pm_Page = (PropertyManagerPage2)swApp.CreatePropertyManagerPage(pageTitle, (int)options, this, ref errors);
//Make sure that the page was created properly
if (errors == (int)swPropertyManagerPageStatus_e.swPropertyManagerPage_Okay)
{
    //Begin adding the controls to the page
    //Create group box
    caption = "Group box";
    options = (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Visible + (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Exp
    pm_Groupbox = (PropertyManagerPageGroup)pm_Page.AddGroupBox(groupboxID, caption, (int)options);

    // Create check box
    control = (int)swPropertyManagerPageControlType_e.swControlType_Checkbox;
    caption = "Focus on text box";
    alignment = (int)swPropertyManagerPageControlLeftAlign_e.swControlAlign_LeftEdge;
    options = (int)swAddControlOptions_e.swControlOptions_Visible + (int)swAddControlOptions_e.swControlOptions_Enabled;
    pm_Checkbox = (PropertyManagerPageCheckbox)pm_Groupbox.AddControl(checkboxID, (short)control, caption, (short)alignm
    Debug.Print("Upon opening the PropertyManager page:");
    Debug.Print(" State of check box: (0 = Unchecked, 1 = Checked, 2 = Indeterminate): " + pm_Checkbox.State);
    pm_Checkbox.Checked = false;

    // Create text box
    control = (int)swPropertyManagerPageControlType_e.swControlType_Textbox;
    caption = "Text box";
    pm_Textbox = (PropertyManagerPageTextbox)pm_Groupbox.AddControl(textboxID, (short)control, caption, (short)alignment
}
else
{
    //If the page is not created
    System.Windows.Forms.MessageBox.Show("An error occurred while attempting to create the PropertyManager page");
}
}

}

#region IPropertyManagerPage2Handler9 Members

void IPropertyManagerPage2Handler9.AfterActivation()
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.AfterClose()
{
    throw new Exception("The method or operation is not implemented.");
}

int IPropertyManagerPage2Handler9.OnActiveXControlCreated(int Id, bool Status)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnButtonPress(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnCheckboxCheck(int Id, bool Checked)
{
    Debug.Print("Upon clicking the check box on the PropertyManager page:");
    Debug.Print(" State of check box: (0 = Unchecked, 1 = Checked, 2 = Indeterminate): " + pm_Checkbox.State);
    // Set focus on the text box when check box is selected
    if (Checked)
    {
        pm_Page.SetFocus(textboxID);
        Debug.Print("Focus on Text box.");
    }
    else
    {
        Debug.Print("Focus off Text box.");
    }
}

void IPropertyManagerPage2Handler9.OnClose(int Reason)
{
    if (Reason == (int)swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Cancel)
    {
        //Do something when the cancel button is clicked
    }
    else if (Reason == (int)swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Okay)
    {
        //Do something else when the OK button is clicked
    }
}

void IPropertyManagerPage2Handler9.OnComboboxEditChanged(int Id, string Text)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnComboboxSelectionChanged(int Id, int Item)
{
    throw new Exception("The method or operation is not implemented.");
}

```

```

void IPropertyManagerPage2Handler9.OnGainedFocus(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnGroupCheck(int Id, bool Checked)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnGroupExpand(int Id, bool Expanded)
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnHelp()
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnKeystroke(int Wparam, int Message, int Lparam, int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnListboxSelectionChanged(int Id, int Item)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnLostFocus(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnNextPage()
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnNumberboxChanged(int Id, double Value)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnOptionCheck(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnPopupMenuItem(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnPopupMenuItemUpdate(int Id, ref int retval)
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnPreview()
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnPreviousPage()
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnRedo()
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSelectionboxCalloutCreated(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSelectionboxCalloutDestroyed(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSelectionboxFocusChanged(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSelectionboxListChanged(int Id, int Count)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSliderPositionChanged(int Id, double Value)

```

```

{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSliderTrackingCompleted(int Id, double Value)
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnSubmitSelection(int Id, object Selection, int SelType, ref string ItemText)
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnTabClicked(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnTextboxChanged(int Id, string Text)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnUndo()
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnWhatsNew()
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnListboxRMBUp(int Id, int PosX, int PosY)
{
    throw new Exception("The method or operation is not implemented.");
}

int IPropertyManagerPage2Handler9.OnWindowFromHandleControlCreated(int Id, bool Status)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnNumberBoxTrackingCompleted(int Id, double Value)
{
    throw new Exception("The method or operation is not implemented.");
}

```

```

#endregion

```

```

}

```

```

}

```

[Back to top](#)

Set Focus on PropertyManager Page Control Example (VBA)

This example shows how to set focus on a PropertyManager page control.

```

'-----
' Preconditions:
' 1. Open a model document.
' 2. Copy Main to the main module.
' 3. Click Insert > Class Module and copy Class1 to that module.
' 4. Click Tools > References > SOLIDWORKS version exposed type libraries
'    for add-in use.
' 5. Open the Immediate window.
'
' Postconditions:
' 1. Creates and displays a PropertyManager page.
' 2. Select the check box to set focus on Text box.
' 3. Examine the Immediate window.
' 4. Clear the check box to remove focus from Text box.
' 5. Examine the Immediate window.
' 6. Click OK to close the PropertyManager page.
'-----
'Main
Option Explicit

Public swApp As SldWorks.SldWorks
Public Part As SldWorks.ModelDoc2
Public pm As Class1

Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc

    'Create a new instance of the PropertyManager class
    Set pm = New Class1
    pm.Show

End Sub

```

[Back to top](#)

```

'Class1

Option Explicit

'Required for PropertyManager page controls
Implements PropertyManagerPage2Handler9

'General objects required for the PropertyManager page
Dim pm_Page As PropertyManagerPage2
Dim pm_Checkbox As PropertyManagerPageCheckbox
Dim pm_Text As PropertyManagerPageTextbox
Dim pm_Group As PropertyManagerPageGroup

'Each object in the page needs a unique ID
Const checkboxID As Integer = 1
Const textID As Integer = 2
Const groupID As Integer = 3

Dim ClickedCancel As Boolean

Sub Show()
    pm_Page.Show2 0
End Sub

'The following runs when a new instance
'of the class is created
Private Sub Class_Initialize()

    Dim PageTitle As String
    Dim options As Long
    Dim errors As Long
    Dim caption As String
    Dim alignment As Long
    Dim control As Long

    'Set the variables for the page
    PageTitle = "Test focus methods"
    options = swPropertyManager_OkayButton + swPropertyManager_CancelButton + swPropertyManagerOptions_LockedPage + swPropertyManagerOptions_PushpinButton

    'Create the PropertyManager page
    Set pm_Page = swApp.CreatePropertyManagerPage(PageTitle, options, Me, errors)

    'Make sure that the page was created properly
    If errors = swPropertyManagerPage_Okay Then
        ' Create group box
        caption = "Group box"
        options = swGroupBoxOptions_Visible + swGroupBoxOptions_Expanded
        Set pm_Group = pm_Page.AddGroupBox(groupID, caption, options)

        ' Create check box
        alignment = swPropertyManagerPageControlLeftAlign_e.swControlAlign_LeftEdge
        options = swAddControlOptions_Visible + swAddControlOptions_e.swControlOptions_Enabled

        control = swControlType_Checkbox
        Set pm_Checkbox = pm_Group.AddControl(checkboxID, control, "Focus on text box", alignment, options, "Check box")
    
```

```

        Debug.Print "Upon opening the PropertyManager page:"
        Debug.Print "   State of check box: (0 = Unchecked, 1 = Checked, 2 = Indeterminate): " & pm_Checkbox.State
        pm_Checkbox.Checked = False

        ' Create text box
        control = swControlType_Textbox
        Set pm_Text = pm_Group.AddControl(textId, control, "Text box", alignment, options, "Text box")

    Else 'If the page is not created
        MsgBox "An error occurred while attempting to create the PropertyManager Page", vbCritical
    End If

End Sub

Private Sub PropertyManagerPage2Handler9_AfterActivation()
End Sub

Private Sub PropertyManagerPage2Handler9_AfterClose()
    'Destroy the class
    Set pm = Nothing
End Sub

Private Function PropertyManagerPage2Handler9_OnActiveXControlCreated(ByVal Id As Long, ByVal Status As Boolean) As Long
End Function

Private Sub PropertyManagerPage2Handler9_OnButtonPress(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnCheckboxCheck(ByVal Id As Long, ByVal Checked As Boolean)
    ' Set focus on the text box when check box is selected
    Debug.Print "Upon clicking the check box on the PropertyManager page:"
    Debug.Print "   State of check box: (0 = Unchecked, 1 = Checked, 2 = Indeterminate): " & pm_Checkbox.State
    If Checked Then
        pm_Page.SetFocus (textId)
        Debug.Print "Focus on Text box."
    Else
        Debug.Print "Focus off Text box."
    End If
End Sub

Private Sub PropertyManagerPage2Handler9_OnClose(ByVal Reason As Long)
    If Reason = swPropertyManagerPageClose_Cancel Then
        'Cancel button was clicked
        ClickedCancel = True
    ElseIf Reason = swPropertyManagerPageClose_Okay Then
        'OK button was clicked
        ClickedCancel = False
    End If
End Sub

Private Sub PropertyManagerPage2Handler9_OnComboboxEditChanged(ByVal Id As Long, ByVal Text As String)
End Sub

Private Sub PropertyManagerPage2Handler9_OnComboboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnGroupCheck(ByVal Id As Long, ByVal Checked As Boolean)
End Sub

Private Sub PropertyManagerPage2Handler9_OnGroupExpand(ByVal Id As Long, ByVal Expanded As Boolean)
End Sub

Private Function PropertyManagerPage2Handler9_OnHelp() As Boolean
End Function

Private Function PropertyManagerPage2Handler9_OnKeystroke(ByVal Wparam As Long, ByVal Message As Long, ByVal Lparam As Long, ByVal Id As Long) As Boolean
End Function

Private Sub PropertyManagerPage2Handler9_OnListboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)
End Sub

Private Function PropertyManagerPage2Handler9_OnNextPage() As Boolean
End Function

Private Sub PropertyManagerPage2Handler9_OnNumberboxChanged(ByVal Id As Long, ByVal Value As Double)
End Sub

Private Sub PropertyManagerPage2Handler9_OnOptionCheck(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnPopupMenuItem(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnPopupMenuItemUpdate(ByVal Id As Long, retVal As Long)
End Sub

Private Function PropertyManagerPage2Handler9_OnPreview() As Boolean
End Function

Private Function PropertyManagerPage2Handler9_OnPreviousPage() As Boolean
End Function

Private Sub PropertyManagerPage2Handler9_OnRedo()
End Sub

```

```

Private Sub PropertyManagerPage2Handler9_OnSelectionboxCalloutCreated(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnSelectionboxCalloutDestroyed(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnSelectionboxFocusChanged(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnSelectionboxListChanged(ByVal Id As Long, ByVal Count As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnSliderPositionChanged(ByVal Id As Long, ByVal Value As Double)
End Sub

Private Sub PropertyManagerPage2Handler9_OnSliderTrackingCompleted(ByVal Id As Long, ByVal Value As Double)
End Sub

Private Function PropertyManagerPage2Handler9_OnSubmitSelection(ByVal Id As Long, ByVal Selection As Object, ByVal SelType As Long, ItemText As String) As Boolean
End Function

Private Function PropertyManagerPage2Handler9_OnTabClicked(ByVal Id As Long) As Boolean
End Function

Private Sub PropertyManagerPage2Handler9_OnTextboxChanged(ByVal Id As Long, ByVal Text As String)
End Sub

Private Sub PropertyManagerPage2Handler9_OnUndo()
End Sub

Private Sub PropertyManagerPage2Handler9_OnWhatsNew()
End Sub

Private Sub PropertyManagerPage2Handler9_OnLostFocus(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnGainedFocus(ByVal Id As Long)
End Sub

Private Function PropertyManagerPage2Handler9_OnWindowFromHandleControlCreated(ByVal Id As Long, ByVal Status As Boolean) As Long
End Function

Private Sub PropertyManagerPage2Handler9_OnListboxRMBUp(ByVal Id As Long, ByVal PosX As Long, ByVal PosY As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnNumberBoxTrackingCompleted(ByVal Id As Long, ByVal Value As Double)
End Sub

End Sub

```

[Back to top](#)

Set Focus on PropertyManager Page Control Example (VB.NET)

This example shows how to set focus on a PropertyManager page control.

```

'-----
' Preconditions:
' 1. Open a model document.
' 2. Copy Main to the main module.
' 3. Click Project > Add Class > Add and copy Class1 to Class1.vb.
' 4. Click Project > Add Reference > browse to install_dir\api\redist,
'    select SolidWorks.Interop.swpublished.dll > OK.
' 5. Open the Immediate window.
'
' Postconditions:
' 1. Creates and displays a PropertyManager page.
' 2. Select the check box to set focus on Text box.
' 3. Examine the Immediate window.
' 4. Clear the check box to remove focus from Text box.
' 5. Examine the Immediate window.
' 6. Click OK to close the PropertyManager page.
'-----
'Main
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Part As ModelDoc2
    Public WithEvents pm As Class1

    Public Sub main()

        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, False)
        Part = swApp.ActiveDoc

        'Create a new instance of the PropertyManager class
        pm = New Class1(swApp)
        pm.Show()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

[Back to top](#)

```

'Class1
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports System
Imports System.Runtime.InteropServices
Imports System.Diagnostics

<ComVisibleAttribute(True)>
Public Class Class1

    Implements PropertyManagerPage2Handler9

    Dim swApp As SldWorks

    'General objects required for the PropertyManager page
    Dim pm_Page As PropertyManagerPage2
    Dim pm_Group As PropertyManagerPageGroup
    Dim pm_Checkbox As PropertyManagerPageCheckbox
    Dim pm_Text As PropertyManagerPageTextbox

    'Each object in the page needs a unique ID
    Const checkboxID As Integer = 1
    Const textID As Integer = 2
    Const groupID As Integer = 3
    Dim ClickedCancel As Boolean

    Sub Show()
        pm_Page.Show2(0)
    End Sub

    'The following runs when a new instance
    'of the class is created
    Public Sub New(ByVal swApp As SldWorks)

        Dim PageTitle As String
        Dim options As Integer
        Dim errors As Integer
        Dim caption As String
        Dim alignment As Integer
        Dim control As Integer

```

```

'Set the variables for the page
PageTitle = "Test focus methods"
options = swPropertyManagerButtonTypes_e.swPropertyManager_OkayButton _
+ swPropertyManagerButtonTypes_e.swPropertyManager_CancelButton _
+ swPropertyManagerPageOptions_e.swPropertyManagerOptions_LockedPage _
+ swPropertyManagerPageOptions_e.swPropertyManagerOptions_PushpinButton

'Create the PropertyManager page
pm_Page = CType(swApp.CreatePropertyManagerPage(PageTitle,
    options, Me, errors), PropertyManagerPage2)

'Make sure that the page was created properly
If errors = swPropertyManagerPageStatus_e.swPropertyManagerPage_Okay Then
    'Begin adding the controls to the page
    ' Create group box
    caption = "Group box"
    options = swAddGroupBoxOptions_e.swGroupBoxOptions_Visible +
        swAddGroupBoxOptions_e.swGroupBoxOptions_Expanded
    pm_Group = pm_Page.AddGroupBox(groupID, caption, options)

    ' Create check box
    alignment = swPropertyManagerPageControlLeftAlign_e.swControlAlign_LeftEdge
    options = swAddControlOptions_e.swControlOptions_Visible +
        swAddControlOptions_e.swControlOptions_Enabled
    control = swPropertyManagerPageControlType_e.swControlType_Checkbox
    pm_Checkbox = pm_Group.AddControl(checkboxID, control, "Focus on text box", alignment, options, "Check box")
    Debug.Print("Upon opening the PropertyManager page:")
    Debug.Print(" State of check box: (0 = Unchecked, 1 = Checked, 2 = Indeterminate): " & pm_Checkbox.State)
    pm_Checkbox.Checked = False

    ' Create text box
    control = swPropertyManagerPageControlType_e.swControlType_Textbox
    pm_Text = pm_Group.AddControl(textID, control, "Text box", alignment, options, "Text box")
Else 'If the page is not created
    MsgBox("An error while attempting to create the PropertyManager Page", vbCritical)
End If

End Sub

Public Sub AfterActivation() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.AfterActivation
End Sub

Public Sub AfterClose() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.AfterClose
End Sub

Public Function OnActiveXControlCreated(ByVal Id As Integer, ByVal Status As Boolean) As Integer Implements SolidWorks.Interop.s
End Function

Public Sub OnButtonPress(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnButtonPr
End Sub

Public Sub OnCheckboxCheck(ByVal Id As Integer, ByVal Checked As Boolean) Implements SolidWorks.Interop.swpublished.IPropertyMar
    Debug.Print("Upon clicking the check box on the PropertyManager page:")
    Debug.Print(" State of check box: (0 = Unchecked, 1 = Checked, 2 = Indeterminate): " & pm_Checkbox.State)
    ' Set focus on the text box when check box is selected
    If Checked Then
        pm_Page.SetFocus(textID)
        Debug.Print("Focus on Text box.")
    Else
        Debug.Print("Focus off Text box.")
    End If
End Sub

Public Sub OnClose(ByVal Reason As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnClose
    If Reason = swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Cancel Then
        ' Cancel button was clicked
        ClickedCancel = True
    ElseIf Reason = swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Okay Then
        ' OK button was clicked
        ClickedCancel = False
    End If
End Sub

Public Sub OnComboboxEditChanged(ByVal Id As Integer, ByVal Text As String) Implements SolidWorks.Interop.swpublished.IPropertyM
End Sub

Public Sub OnComboboxSelectionChanged(ByVal Id As Integer, ByVal Item As Integer) Implements SolidWorks.Interop.swpublished.IPro
End Sub

Public Sub OnGainedFocus(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnGainedFo
End Sub

Public Sub OnGroupCheck(ByVal Id As Integer, ByVal Checked As Boolean) Implements SolidWorks.Interop.swpublished.IPropertyManag
End Sub

Public Sub OnGroupExpand(ByVal Id As Integer, ByVal Expanded As Boolean) Implements SolidWorks.Interop.swpublished.IPropertyMan
End Sub

Public Function OnHelp() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnHelp
End Function

Public Function OnKeystroke(ByVal Wparam As Integer, ByVal Message As Integer, ByVal Lparam As Integer, ByVal Id As Integer) As
End Function

Public Sub OnListboxSelectionChanged(ByVal Id As Integer, ByVal Item As Integer) Implements SolidWorks.Interop.swpublished.IProp
End Sub

```

```

Public Sub OnLostFocus(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnLostFocus
End Sub

Public Function OnNextPage() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnNextPage
End Function

Public Sub OnNumberboxChanged(ByVal Id As Integer, ByVal Value As Double) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnNumberboxChanged
End Sub

Public Sub OnOptionCheck(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnOptionCheck
End Sub

Public Sub OnPopupMenuItemClick(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnPopupMenuItemClick
End Sub

Public Sub OnPopupMenuItemUpdate(ByVal Id As Integer, ByRef retval As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnPopupMenuItemUpdate
End Sub

Public Function OnPreview() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnPreview
End Function

Public Function OnPreviousPage() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnPreviousPage
End Function

Public Sub OnRedo() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnRedo
End Sub

Public Sub OnSelectionboxCalloutCreated(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSelectionboxCalloutCreated
End Sub

Public Sub OnSelectionboxCalloutDestroyed(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSelectionboxCalloutDestroyed
End Sub

Public Sub OnSelectionboxFocusChanged(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSelectionboxFocusChanged
End Sub

Public Sub OnSelectionboxListChanged(ByVal Id As Integer, ByVal Count As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSelectionboxListChanged
End Sub

Public Sub OnSliderPositionChanged(ByVal Id As Integer, ByVal Value As Double) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSliderPositionChanged
End Sub

Public Sub OnSliderTrackingCompleted(ByVal Id As Integer, ByVal Value As Double) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSliderTrackingCompleted
End Sub

Public Function OnSubmitSelection(ByVal Id As Integer, ByVal Selection As Object, ByVal SelType As Integer, ByRef ItemText As String) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSubmitSelection
End Function

Public Function OnTabClicked(ByVal Id As Integer) As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnTabClicked
End Function

Public Sub OnTextboxChanged(ByVal Id As Integer, ByVal Text As String) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnTextboxChanged
End Sub

Public Sub OnUndo() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnUndo
End Sub

Public Sub OnWhatsNew() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnWhatsNew
End Sub

Public Sub OnListBoxRMBUp(ByVal Id As Integer, ByVal posX As Integer, ByVal posY As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnListBoxRMBUp
End Sub

Public Function OnWindowFromHandleControlCreated(ByVal Id As Integer, ByVal Status As Boolean) As Integer Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnWindowFromHandleControlCreated
End Function

Public Sub OnNumberBoxTrackingCompleted(ByVal Id As Integer, ByVal Value As Double) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnNumberBoxTrackingCompleted
End Sub

```

End Class

[Back to top](#)

Set Fully Resolved Assembly to Lightweight Example (C#)

This example shows how to set a fully resolved assembly to lightweight.

```
//-----
// Preconditions:
// 1. Open a fully resolved assembly document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Sets all assembly components to lightweight.
// 2. Examine the Immediate window and FeatureManager design tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace SetSuppression2.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssy = default(AssemblyDoc);
            Configuration swConfig = default(Configuration);
            ConfigurationManager swConfigMgr = default(ConfigurationManager);
            Component2 swRootComp = default(Component2);
            bool bRet = false;
            string sPadStr = " ";

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swConfigMgr = (ConfigurationManager)swModel.ConfigurationManager;
            swConfig = (Configuration)swConfigMgr.ActiveConfiguration;
            swRootComp = (Component2)swConfig.GetRootComponent3(true);
            Debug.Print("File = " + swModel.GetPathName());
            SetComponentLightWeight(sPadStr, swRootComp);

            // Update in-context features
            bRet = swModel.ForceRebuild3(false);
            Debug.Assert(bRet);
        }

        public void SetComponentLightWeight(string sPadStr, Component2 swComp)
        {
            object[] vChildArr = null;
            Component2 swChildComp = default(Component2);
            ModelDoc2 swChildModel = default(ModelDoc2);
            int nRetVal = 0;
            int nDocType = 0;
            int i = 0;

            Debug.Print(sPadStr + swComp.Name2 + " [" + Convert.ToBoolean(swComp.Visible) + "]");
            vChildArr = (object[])swComp.GetChildren();
            for (i = 0; i < vChildArr.Length; i++)
            {
                swChildComp = (Component2)vChildArr[i];
                // Is null if another instance has been previously set to lightweight
                swChildModel = (ModelDoc2)swChildComp.GetModelDoc2();
                if ((swChildModel != null))
                {
                    nDocType = swChildModel.GetType();
                }
                else
                {
                    nDocType = (int)swDocumentTypes_e.swDocNONE;
                }
                nRetVal = swChildComp.SetSuppression2((int)swComponentSuppressionState_e.swComponentLightweight);
                if ((int)swDocumentTypes_e.swDocPART == nDocType | (int)swDocumentTypes_e.swDocNONE == nDocType)
                {
                    Debug.Assert((int)swComponentResolveStatus_e.swResolveNotPerformed == nRetVal);
                }
                else
                {
                    // Cannot set a sub-assembly to lightweight; must set each part to lightweight
                    Debug.Assert((int)swDocumentTypes_e.swDocASSEMBLY == swChildModel.GetType());
                    Debug.Assert((int)swComponentResolveStatus_e.swResolveError == nRetVal);
                }
                // Recurse into this component
                SetComponentLightWeight(sPadStr + " ", swChildComp);
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Set Fully Resolved Assembly to Lightweight Example (VBA)

This example shows how to set a fully resolved assembly to lightweight.

```

'-----
' Preconditions:
' 1. Open a fully resolved assembly document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Sets all assembly components to lightweight.
' 2. Examine the Immediate window and FeatureManager design tree.
'-----

Sub SetComponentLightWeight(sPadStr As String, swComp As SldWorks.Component2)

    Dim vChildArr As Variant
    Dim swChildComp As SldWorks.Component2
    Dim swChildModel As SldWorks.ModelDoc2
    Dim nRetVal As Long
    Dim nDocType As Long
    Dim i As Long

    Debug.Print sPadStr & swComp.Name2 & " [" & CBool(swComp.Visible) & "]"

    vChildArr = swComp.Children
    For i = 0 To UBound(vChildArr)
        Set swChildComp = vChildArr(i)
        ' Is Nothing if another instance has been previously set to lightweight
        Set swChildModel = swChildComp.ModelDoc2
        If Not swChildModel Is Nothing Then
            nDocType = swChildModel.GetType
        Else
            nDocType = swDocNONE
        End If

        nRetVal = swChildComp.SetSuppression2(swComponentLightweight)
        If swDocPART = nDocType Or swDocNONE = nDocType Then
            Debug.Assert swResolveNotPerformed = nRetVal
        Else
            ' Cannot set a sub-assembly to lightweight; must set each part to lightweight
            Debug.Assert swDocASSEMBLY = swChildModel.GetType
            Debug.Assert swResolveError = nRetVal
        End If

        ' Recurse into this component
        SetComponentLightWeight sPadStr & " ", swChildComp
    Next i
End Sub

Sub main()

    Dim swApp                As SldWorks.SldWorks
    Dim swModel               As SldWorks.ModelDoc2
    Dim swAssy                As SldWorks.AssemblyDoc
    Dim swConfig              As SldWorks.Configuration
    Dim swConfigMgr           As SldWorks.ConfigurationManager
    Dim swRootComp            As SldWorks.Component2
    Dim bRet                  As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swConfigMgr = swModel.ConfigurationManager
    Set swConfig = swConfigMgr.ActiveConfiguration
    Set swRootComp = swConfig.GetRootComponent3(True)
    Debug.Print "File = " & swModel.GetPathName
    SetComponentLightWeight " ", swRootComp

    ' Update in-context features
    bRet = swModel.ForceRebuild3(False): Debug.Assert bRet
End Sub

```

Set Fully Resolved Assembly to Lightweight Example (VB.NET)

This example shows how to set a fully resolved assembly to lightweight.

```

-----
Preconditions:
1. Open a fully resolved assembly document.
2. Open the Immediate window.
-----
Postconditions:
1. Sets all assembly components to lightweight.
2. Examine the Immediate window and FeatureManager design tree.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swAssy As AssemblyDoc
        Dim swConfig As Configuration
        Dim swConfigMgr As ConfigurationManager
        Dim swRootComp As Component2
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc
        swAssy = swModel
        swConfigMgr = swModel.ConfigurationManager
        swConfig = swConfigMgr.ActiveConfiguration
        swRootComp = swConfig.GetRootComponent3(True)
        Debug.Print("File = " & swModel.GetPathName)
        SetComponentLightWeight("", swRootComp)

        ' Update in-context features
        bRet = swModel.ForceRebuild3(False) : Debug.Assert(bRet)

    End Sub

    Sub SetComponentLightWeight(ByVal sPadStr As String, ByVal swComp As Component2)
        Dim vChildArr As Object
        Dim swChildComp As Component2
        Dim swChildModel As ModelDoc2
        Dim nRetVal As Integer
        Dim nDocType As Integer
        Dim i As Integer

        Debug.Print(sPadStr & swComp.Name2 & " [" & CBool(swComp.Visible) & "]")
        vChildArr = swComp.Children
        For i = 0 To UBound(vChildArr)
            swChildComp = vChildArr(i)
            ' Is Nothing if another instance has been previously set to lightweight
            swChildModel = swChildComp.GetModelDoc2
            If Not swChildModel Is Nothing Then
                nDocType = swChildModel.GetType
            Else
                nDocType = swDocumentTypes_e.swDocNONE
            End If
            nRetVal = swChildComp.SetSuppression2(swComponentSuppressionState_e.swComponentLightweight)
            If swDocumentTypes_e.swDocPART = nDocType Or swDocumentTypes_e.swDocNONE = nDocType Then
                Debug.Assert(swComponentResolveStatus_e.swResolveNotPerformed = nRetVal)
            Else
                ' Cannot set a sub-assembly to lightweight; must set each part to lightweight
                Debug.Assert(swDocumentTypes_e.swDocASSEMBLY = swChildModel.GetType)
                Debug.Assert(swComponentResolveStatus_e.swResolveError = nRetVal)
            End If
            ' Recurse into this component
            SetComponentLightWeight(sPadStr & " ", swChildComp)
        Next i
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Set Grid Lines Example (VBA)

This example shows how to set the number of minor grid lines between major grid lines in an active sketch.

```
'-----
' Preconditions:
' 1. Open a part document.
' 2. Open a sketch in the part document.
' 3. If the grid is not displayed in the sketch,
'    click View > Hide/Show > Grid.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Gets the number of minor grid lines between
'    the major grid lines.
' 2. Sets the number of minor grid lines between
'    major grid lines to 15.
' 3. Gets the number of minor grid lines between
'    the major grid lines.
' 4. Examine the graphics area and Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim bReturnValue As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Debug.Print "Current minor grid lines between major grid lines: " & swModel.GetUserPreferenceIntegerValue(swGridMinorLinesPerMajor)
    bReturnValue = swModel.SetUserPreferenceIntegerValue(swGridMinorLinesPerMajor, 15)
    Debug.Print "Modified minor grid lines between major grid lines: " & swModel.GetUserPreferenceIntegerValue(swGridMinorLinesPerMajor)

End Sub
```

Set Independent Viewport for MBD 3D PDF Example (C#)

This example shows how to set a view for an independent viewport for a SOLIDWORKS MBD 3D PDF.

```
//-----
// Preconditions:
// 1. Open an assembly.
// 2. Copy install_dir\data\themes\simple assembly (letter, landscape) to
//    C:\test\simple assembly (letter, landscape).
// 3. In SOLIDWORKS Design, click the SOLIDWORKS MBD toolbar > 3D PDF Template Editor.
//    a. Click Independent Viewport.
//    b. Click Save As.
//    c. Ensure that Theme Directory is C:\test\simple assembly (letter, landscape).
//    d. Type AllViews in Name.
//    e. Click Save.
//    f. Close the 3D PDF Template Editor.
//
// Postconditions:
// 1. Gets the MBD3DPdfData object.
// 2. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.
// 3. Sets to display SOLIDWORKS MBD 3D PDF after publishing.
// 4. Sets the theme for the SOLIDWORKS MBD 3D PDF.
// 5. Sets the view to display in the independent viewport
//    for the SOLIDWORKS MBD 3D PDF.
// 6. Creates and displays MBDPart1.PDF.
// 7. Examine MBDPart1.PDF.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            MBD3DPdfData swMBDPdfData = default(MBD3DPdfData);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Get MBD3DPdfData object
            swMBDPdfData = (MBD3DPdfData)swModelDocExt.GetMBD3DPdfData();

            //Set path and file name for SOLIDWORKS MBD 3D PDF
            swMBDPdfData.FilePath = "C:\\temp\\MBDPart1.PDF";

            //Display SOLIDWORKS MBD 3D PDF
            swMBDPdfData.ViewPdfAfterSaving = true;

            //Set SOLIDWORKS MBD 3D PDF theme
            swMBDPdfData.ThemeName = "C:\\test\\simple assembly (letter, landscape)\\AllViews\\theme.xml";

            //Set view for AllViews
            swMBDPdfData.SetIndependentViewPort("*Top");

            //Create SOLIDWORKS MBD 3D PDF
            swModelDocExt.PublishTo3DPDF(swMBDPdfData);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Set Independent Viewport for MBD 3D PDF Example (VBA)

This example shows how to set a view for an independent viewport for a SOLIDWORKS MBD 3D PDF.

```
'-----
' Preconditions:
' 1. Open an assembly.
' 2. Copy install_dir\data\themes\simple assembly (letter, landscape) to
'   C:\test\simple assembly (letter, landscape).
' 3. In SOLIDWORKS Design, click the SOLIDWORKS MBD toolbar > 3D PDF Template Editor.
'   a. Click Independent Viewport.
'   b. Click Save As.
'   c. Ensure that Theme Directory is C:\test\simple assembly (letter, landscape).
'   d. Type AllViews in Name.
'   e. Click Save.
'   f. Close the 3D PDF Template Editor.
'
' Postconditions:
' 1. Gets the MBD3DPdfData object.
' 2. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.
' 3. Sets to display SOLIDWORKS MBD 3D PDF after publishing.
' 4. Sets the theme for the SOLIDWORKS MBD 3D PDF.
' 5. Sets the view to display in the independent viewport
'    for the SOLIDWORKS MBD 3D PDF.
' 6. Creates and displays MBDPart1.PDF.
' 7. Examine MBDPart1.PDF.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swMBDPdfData As SldWorks.MBD3DPdfData
```

Sub main()

```
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
```

```
    'Get MBD3DPdfData object
    Set swMBDPdfData = swModelDocExt.GetMBD3DPdfData
```

```
    'Set path and file name for SOLIDWORKS MBD 3D PDF
    swMBDPdfData.FilePath = "C:\temp\MBDPart1.PDF"
```

```
    'Display SOLIDWORKS MBD 3D PDF
    swMBDPdfData.ViewPdfAfterSaving = True
```

```
    'Set SOLIDWORKS MBD 3D PDF theme
    swMBDPdfData.ThemeName = "C:\test\simple assembly (letter, landscape)\AllViews\theme.xml"
```

```
    'Set view for AllViews
    swMBDPdfData.SetIndependentViewPort "*Top"
```

```
    'Create SOLIDWORKS MBD 3D PDF
    swModelDocExt.PublishTo3DPDF swMBDPdfData
```

End Sub

Set_Independent_Viewport_for_MBD_3D_PDF_Example_VBNET (Topic)

SOLIDWORKS API Help

Set Independent Viewport for MBD 3D PDF Example (VB.NET)

This example shows how to set a view for an independent viewport for a SOLIDWORKS MBD 3D PDF.

```
'-----
' Preconditions:
' 1. Open an assembly.
' 2. Copy install_dir\data\themes\simple assembly (letter, landscape) to
'    C:\test\simple assembly (letter, landscape).
' 3. In SOLIDWORKS Design, click the SOLIDWORKS MBD toolbar > 3D PDF Template Editor.
'    a. Click Independent Viewport.
'    b. Click Save As.
'    c. Ensure that Theme Directory is C:\test\simple assembly (letter, landscape).
'    d. Type AllViews in Name.
'    e. Click Save.
'    f. Close the 3D PDF Template Editor.
'
' Postconditions:
' 1. Gets the MBD3DPdfData object.
' 2. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.
' 3. Sets to display SOLIDWORKS MBD 3D PDF after publishing.
' 4. Sets the theme for the SOLIDWORKS MBD 3D PDF.
' 5. Sets the view to display in the independent viewport
'    for the SOLIDWORKS MBD 3D PDF.
' 6. Creates and displays MBDPart1.PDF.
' 7. Examine MBDPart1.PDF.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swMBDPdfData As MBD3DPdfData

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        'Get MBD3DPdfData object
        swMBDPdfData = swModelDocExt.GetMBD3DPdfData

        'Set path and file name for SOLIDWORKS MBD 3D PDF
        swMBDPdfData.FilePath = "C:\temp\MBDPart1.PDF"

        'Display SOLIDWORKS MBD 3D PDF
        swMBDPdfData.ViewPdfAfterSaving = True

        'Set SOLIDWORKS MBD 3D PDF theme
        swMBDPdfData.ThemeName = "C:\test\simple assembly (letter, landscape)\AllViews\theme.xml"

        'Set view for AllViews
        swMBDPdfData.SetIndependentViewPort("**Top")

        'Create SOLIDWORKS MBD 3D PDF
        swModelDocExt.PublishTo3DPDF(swMBDPdfData)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

End Class

Set Material Example (C#)

This example shows how to get the names of the material schema, material databases, and bodies in a part document. This example also shows how to apply a SOLIDWORKS Design Material to all of the bodies in a part document.

```
//-----
// Preconditions:
// 1. Verify that the specified document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Applies the material ABS PC (polycarbonate plastic)
//    from the Material database to all
//    bodies in the part.
// 3. To verify, examine:
//    * DisplayManager tab
//    * graphics area
//    * Immediate window
//
// NOTE: Because the part document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace GetMaterialBodyCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            PartDoc swPart = default(PartDoc);
            Body2 swBody = default(Body2);
            int errors = 0;
            int warnings = 0;
            object[] vMatDBarr = null;
            object[] vMatDB = null;
            object[] Bodies = null;
            int BodyMaterialError = 0;
            string sMatName = "";
            string sMatDB = "";
            int i = 0;
            int j = 0;

            // Open the document
            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\multibody\\multi_inter.sldprt", (int)swDocumentTypes_e.swDocPART,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swPart = (PartDoc)swModel;
            // Get the material schema and names
            // of available materials databases
            vMatDBarr = (object[])swApp.GetMaterialDatabases();
            Debug.Print("Material schema pathname = " + swApp.GetMaterialSchemaPathName());
            for (i=0; i < vMatDBarr.Length; i++)
            {
                Debug.Print(" Material database: " + vMatDB);
            }
            Debug.Print("");
            Bodies = (object[])swPart.GetBodies2((int)swBodyType_e.swAllBodies, false);
            for (j = 0; j < Bodies.Length; j++)
            {
                swBody = (Body2)Bodies[j];
                // Get the name of the body
                Debug.Print(swBody.Name);

                swBody.Select2(false, null);

                // Set the SOLIDWORKS Design material for that body
                BodyMaterialError = swBody.SetMaterialProperty("Default", "solidworks
materials.sldmat", "ABS PC");
                // Comment out the previous statement and uncomment the following statement to use
                custom material
                //BodyMaterialError = swBody.SetMaterialProperty("Default", "custom materials.sldmat",
                "Custom Plastic");

                // Get the names of the body's material and the
                // database to which it belongs
                sMatName = swBody.GetMaterialPropertyName("", out sMatDB);
                if (string.IsNullOrEmpty(sMatName))
                {
                    Debug.Print("Body " + j + "'s material name: No material applied");
                }
                else
                {
                    Debug.Print("Body " + j + "'s material name: " + sMatName);
                }
            }
        }
    }
}
```

```
        Debug.Print("Body " + j + "'s material database: " + sMatDB);
        Debug.Print(" ");
    }
}
}
/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}
```

Set Material Example (VBA)

This example shows how to get the names of the material schema, material databases, and bodies in a part document. This example also shows how to apply a SOLIDWORKS Design Material to all of the bodies in a part document.

```
'-----
' Preconditions:
' 1. Verify that the specified document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Applies the material ABS PC (polycarbonate plastic)
'    from the Material database to all
'    bodies in the part.
' 3. To verify, examine:
'    * DisplayManager tab
'    * graphics area
'    * Immediate window
'
' NOTE: Because the part document is used elsewhere,
' do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim swBody As SldWorks.Body2
    Dim errors As Long
    Dim warnings As Long
    Dim vMatDBarr As Variant
    Dim vMatDB As Variant
    Dim Bodies As Variant
    Dim BodyMaterialError As Long
    Dim sMatName As String
    Dim sMatDB As String
    Dim itr As Long
    Dim boolstat As Boolean

    Set swApp = Application.SldWorks

    ' Open the document
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt", swDocPART,
swOpenDocOptions_Silent, "", errors, warnings)
    Set swPart = swModel

    ' Get the material schema and names
    ' of available materials databases
    vMatDBarr = swApp.GetMaterialDatabases
    Debug.Print "Material schema pathname = " & swApp.GetMaterialSchemaPathName
    For Each vMatDB In vMatDBarr
        Debug.Print "    Material database: " & vMatDB
    Next

    Debug.Print ""

    Bodies = swPart.GetBodies2(swAllBodies, False)
    For itr = 0 To UBound(Bodies)
        Set swBody = Bodies(itr)
        ' Get the name of the body
        Debug.Print swBody.Name
        boolstat = swBody.Select2(False, Nothing)
        ' Set the SOLIDWORKS Design material for that body
        BodyMaterialError = swBody.SetMaterialProperty("Default", "solidworks materials.sldmat", "ABS PC")

        ' Comment out previous statement and uncomment following statement to use custom material
        'BodyMaterialError = swBody.SetMaterialProperty("Default", "custom materials.sldmat", "Custom Plastic")

        ' Get the names of the body's material and the
        ' database to which it belongs
        sMatName = swBody.GetMaterialPropertyName("", sMatDB)
        If sMatName = "" Then
            Debug.Print ("Body " & itr & "'s material name: No material applied")
        Else
            Debug.Print ("Body " & itr & "'s material name: " & sMatName)
            Debug.Print ("Body " & itr & "'s material database: " & sMatDB)
            Debug.Print (" ")
        End If
    Next itr

End Sub
```

Set Material Example (VB.NET)

This example shows how to get the names of the material schema, material databases, and bodies in a part document. This example also shows how to apply a SOLIDWORKS Design Material to all of the bodies in a part document.

```
'-----
' Preconditions:
' 1. Verify that the specified document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Applies the material ABS PC (polycarbonate plastic)
'    from the Material database to all
'    bodies in the part.
' 3. To verify, examine:
'    * DisplayManager tab
'    * graphics area
'    * Immediate window
'
' NOTE: Because the part document is used elsewhere,
' do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swPart As PartDoc
        Dim swBody As Body2
        Dim errors As Integer
        Dim warnings As Integer
        Dim vMatDBarr As Object
        Dim vMatDB As Object
        Dim Bodies As Object
        Dim BodyMaterialError As Integer
        Dim sMatName As String = ""
        Dim sMatDB As String = ""
        Dim itr As Integer
        Dim boolstat As Boolean

        ' Open the document
        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\multibody\multi_inter.sldprt", swDocumentTypes_e.swDocPART,
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swPart = swModel

        ' Get the material schema and names
        ' of available materials databases
        vMatDBarr = swApp.GetMaterialDatabases
        Debug.Print("Material schema pathname = " & swApp.GetMaterialSchemaPathName)
        For Each vMatDB In vMatDBarr
            Debug.Print("  Material database: " & vMatDB)
        Next

        Debug.Print("")

        Bodies = swPart.GetBodies2(swBodyType_e.swAllBodies, False)
        For itr = 0 To UBound(Bodies)
            swBody = Bodies(itr)
            ' Get the name of the body
            Debug.Print(swBody.Name)

            boolstat = swBody.Select2(False, Nothing)

            ' Set the SOLIDWORKS Design material for that body
            BodyMaterialError = swBody.SetMaterialProperty("Default", "solidworks materials.sldmat",
"ABS PC")
            ' Comment out previous statement and uncomment following statement to use custom material
            'BodyMaterialError = swBody.SetMaterialProperty("Default", "custom materials.sldmat",
"Custom Plastic")

            ' Get the names of the body's material and the
            ' database to which it belongs
            sMatName = swBody.GetMaterialPropertyName("", sMatDB)
            If sMatName = "" Then
                Debug.Print("Body " & itr & "'s material name: No material applied")
            Else
                Debug.Print("Body " & itr & "'s material name: " & sMatName)
            End If
        Next
    End Sub
End Class
```

```
        Debug.Print("Body " & itr & "'s material database: " & sMatDB)
        Debug.Print(" ")
    End If
Next itr
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Set Material Property Name Example (VBA)

This example shows how to set a part's material property name.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Changes the model's and part's material property name  
'    Alloy Steel.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swPart As SldWorks.PartDoc  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "  Old model material   = " & swModel.MaterialIdName  
    Debug.Print "  Old part  material    = " & swPart.MaterialIdName  
  
    swPart.SetMaterialPropertyName "SOLIDWORKS Materials.sldmat", "Alloy Steel"  
  
    Debug.Print "  New model material   = " & swModel.MaterialIdName  
    Debug.Print "  New part  material    = " & swPart.MaterialIdName  
  
End Sub
```

Set Model Display Mode Example (VBA)

This example shows how to set the display mode of a model view.

```
'-----  
' Preconditions:  
' 1. Open a part or assembly document.  
' 2. Start up a ray-trace rendering engine.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Sets the model view display to integrated preview.  
' 2. Examine the graphics area and Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Const nNewDispMode As Long = swViewDisplayMode_e.swViewDisplayMode_IntegratedPreview  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModVie As SldWorks.ModelView  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModView = swModel.ActiveView  
  
    swModView.DisplayMode = nNewDispMode  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "  Display mode (swViewDisplayMode_e.swViewDisplayMode_IntegratedPreview = 13): " & swModView.DisplayMode  
  
End Sub
```

Set New End Condition for Simple Hole Feature Example (C#)

This example shows how to set a new end condition for a simple hole feature.

```
//-----
// Preconditions:
// 1. Open a part document containing a block with a simple hole feature named
//    Hole1.
// 2. Select the bottom face of the block.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Sets the bottom face of the block as the new end condition
//    of the simple hole feature.
// 2. Examine the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace SimpleHoleFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Model;
        SelectionMgr SelMgr;
        Feature SimpleHoleFeature;
        SimpleHoleFeatureData2 SimpleHoleFeature_DEF;
        Face2 bottomFace;
        Face2 SimpleHoleEndCondition;

        bool boolstatus;

        public void Main()
        {
            Model = (ModelDoc2)swApp.ActiveDoc;
            SelMgr = (SelectionMgr)Model.SelectionManager;

            bottomFace = (Face2)SelMgr.GetSelectedObject6(1, -1);

            boolstatus = Model.Extension.SelectByID2("Hole1", "BODYFEATURE", 0, 0, 0, false, 0, null,
(int)swSelectOption_e.swSelectOptionDefault);
            SimpleHoleFeature = (Feature)SelMgr.GetSelectedObject6(1, -1);

            SimpleHoleFeature_DEF = (SimpleHoleFeatureData2)SimpleHoleFeature.GetDefinition();
            SimpleHoleFeature_DEF.AccessSelections(Model, null);

            int SimpleHoleType = 0;

            // Print the type of hole
            SimpleHoleType = SimpleHoleFeature_DEF.Type;
            Debug.Print("Hole type: " + SimpleHoleType);

            // If the end condition is blind, no reference is returned
            SimpleHoleEndCondition = (Face2)SimpleHoleFeature_DEF.GetEndConditionReference(out SimpleHoleType);

            SimpleHoleFeature_DEF.Type = (int)swEndConditions_e.swEndCondUpToSurface;
            SimpleHoleFeature_DEF.SetEndConditionReference(bottomFace);
            SimpleHoleFeature.ModifyDefinition(SimpleHoleFeature_DEF, Model, null);
            SimpleHoleFeature_DEF.AccessSelections(Model, null);

            // Print the type of hole
            SimpleHoleType = SimpleHoleFeature_DEF.Type;
            Debug.Print("Hole type after setting end condition: " + SimpleHoleType);

            // The end condition is not blind, so a reference is returned
            SimpleHoleEndCondition = (Face2)SimpleHoleFeature_DEF.GetEndConditionReference(out SimpleHoleType);
            Debug.Print("End condition face ID: " + SimpleHoleEndCondition.GetFaceId());

            SimpleHoleFeature_DEF.ReleaseSelectionAccess();
        }

        public SldWorks swApp;
    }
}
```

Set New End Condition for Simple Hole Feature Example (VB.NET)

This example shows how to set a new end condition for a simple hole feature.

```

'-----
' Preconditions:
' 1. Open a part document containing a block with a simple hole feature named
'    Hole1.
' 2. Select the bottom face of the block.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Sets the bottom face of the block as the new end condition
'    of the simple hole feature.
' 2. Examine the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim Model As ModelDoc2
    Dim SelMgr As SelectionMgr
    Dim SimpleHoleFeature As Feature
    Dim SimpleHoleFeature_DEF As SimpleHoleFeatureData2
    Dim bottomFace As Face2
    Dim SimpleHoleEndCondition As Face2
    Dim boolstatus As Boolean

```

```
Sub main()
```

```

    Model = swApp.ActiveDoc
    SelMgr = Model.SelectionManager

```

```
    bottomFace = SelMgr.GetSelectedObject6(1, -1)
```

```

    boolstatus = Model.Extension.SelectByID("Hole1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing,
swSelectOption_e.swSelectOptionDefault)
    SimpleHoleFeature = SelMgr.GetSelectedObject6(1, -1)

```

```

    SimpleHoleFeature_DEF = SimpleHoleFeature.GetDefinition
    SimpleHoleFeature_DEF.AccessSelections(Model, Nothing)

```

```
Dim SimpleHoleType As Integer
```

```

' Print the type of hole
SimpleHoleType = SimpleHoleFeature_DEF.Type
Debug.Print("Hole type: " & SimpleHoleType)

```

```

' If the end condition is blind, no reference is returned
SimpleHoleEndCondition = SimpleHoleFeature_DEF.GetEndConditionReference(SimpleHoleType)

```

```

SimpleHoleFeature_DEF.Type = swEndConditions_e.swEndCondUpToSurface
SimpleHoleFeature_DEF.SetEndConditionReference(bottomFace)
SimpleHoleFeature.ModifyDefinition(SimpleHoleFeature_DEF, Model, Nothing)
SimpleHoleFeature_DEF.AccessSelections(Model, Nothing)

```

```

' Print the type of hole
SimpleHoleType = SimpleHoleFeature_DEF.Type
Debug.Print("Hole type after setting end condition: " & SimpleHoleType)

```

```

' The end condition is not blind, so a reference is returned
SimpleHoleEndCondition = SimpleHoleFeature_DEF.GetEndConditionReference(SimpleHoleType)

```

```
SimpleHoleFeature_DEF.ReleaseSelectionAccess()
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Set New End Condition for Simple Hole Feature Example (VBA)

This example shows how to set a new end condition for a simple hole feature.

```

'-----
' Preconditions:
' 1. Open a part document containing a block with a simple hole feature named
'    Hole1.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Select the bottom face of the block at Stop, then press F5.
' 2. Sets the bottom face of the block as the new end condition
'    of the simple hole feature.
' 3. Examine the Immediate window.
'-----

Dim swApp As SldWorks.SldWorks
Dim Model As SldWorks.ModelDoc2
Dim SelMgr As SldWorks.SelectionMgr
Dim SimpleHoleFeature As SldWorks.Feature
Dim SimpleHoleFeature_DEF As SldWorks.SimpleHoleFeatureData2
Dim bottomFace As SldWorks.Face2
Dim SimpleHoleEndCondition As SldWorks.Face2
Dim boolstatus As Boolean

Option Explicit

Sub main()
    Set swApp = Application.SldWorks
    Set Model = swApp.ActiveDoc
    Set SelMgr = Model.SelectionManager

    boolstatus = Model.Extension.SelectByID2("Hole1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    Set SimpleHoleFeature = SelMgr.GetSelectedObject6(1, -1)

    Stop 'Select the bottom face of the block, then press F5
    Set bottomFace = SelMgr.GetSelectedObject6(1, -1)

    Set SimpleHoleFeature_DEF = SimpleHoleFeature.GetDefinition
    SimpleHoleFeature_DEF.AccessSelections Model, Nothing

    Dim SimpleHoleType As Long

    ' Print the type of hole
    SimpleHoleType = SimpleHoleFeature_DEF.Type
    Debug.Print "Hole type: " & SimpleHoleType

    ' If the end condition is blind, no reference is returned
    Set SimpleHoleEndCondition = SimpleHoleFeature_DEF.GetEndConditionReference(SimpleHoleType)

    SimpleHoleFeature_DEF.Type = swEndCondUpToSurface
    SimpleHoleFeature_DEF.SetEndConditionReference bottomFace
    SimpleHoleFeature.ModifyDefinition SimpleHoleFeature_DEF, Model, Nothing
    SimpleHoleFeature_DEF.AccessSelections Model, Nothing

    ' Print the type of hole
    SimpleHoleType = SimpleHoleFeature_DEF.Type
    Debug.Print "Hole type after setting end condition: " & SimpleHoleType

    ' The end condition is not blind, so a reference is returned
    Set SimpleHoleEndCondition = SimpleHoleFeature_DEF.GetEndConditionReference(SimpleHoleType)

    SimpleHoleFeature_DEF.ReleaseSelectionAccess
End Sub

```

Set_Note_Name_Example (Topic)

SOLIDWORKS API Help

Set Note Name Example (VBA)

This example shows how to set the name of a selected note. You can combine this code with a dialog that prompts the user for a desired name, or you can programmatically determine the new note name.

```
'-----
Sub SetNoteName( newNoteName As String )

    ' Define variable used to hold the SldWorks object
    Dim swApp As Object

    ' Define variable used to hold the ModelDoc object
    Dim Model As Object

    ' Define variable used to hold the SelectionMgr object
    Dim SelMgr As Object
    Dim note As Object
    Dim checkNoteName As String

    Const swSelNOTES = 15
    Const swDocDRAWING = 3

    ' Attach to or open SOLIDWORKS Design session
    Set swApp = CreateObject("Sldworks.Application")
    ' Grab the current document
    Set Model = swApp.ActiveDoc

    ' Verify that the document is a drawing
    If (Model Is Nothing) Or (Model.GetType <> swDocDRAWING) Then
        Exit Sub
    End If

    Set SelMgr = Model.SelectionManager

    ' Make sure a selection exists
    If (SelMgr.GetSelectedObjectCount = 0) Then
        swApp.SendMsgToUser "Select a Note first..."
    Else
        ' If selected object is a note
        If (SelMgr.GetSelectedObjectType(1) = 15) Then
            ' Get the Note object from the selection list
            Set note = SelMgr.GetSelectedObject2(1)
            ' Use the Note object to change the note name
            res = note.SetName (newNoteName)
            checkNoteName = note.GetName

            ' If note name does not match
            If (checkNoteName <> newNoteName) Then
                swApp.SendMsgToUser "Error changing note name."
            End If
        End If
    End If

End Sub
```

Set Override Option for Auto Relief Default Parameters Example (C#)

This example shows how to set the override option for the auto relief default parameters of a sheet metal feature in a multibody sheet metal part.

```
//-----
// Preconditions:
// 1. Open a multibody sheet metal part that contains
//   a Sheet-Metal3 feature in the Sheet-Metal folder.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets whether the default parameters are overridden
//   for Sheet-Metal3.
// 2. Sets the override option for the auto relief default
//   parameters of Sheet-Metal3.
// 3. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            SheetMetalFeatureData swSheetMetalFeatureData = default(SheetMetalFeatureData);
            int errors = 0;
            bool status = false;
            bool overridden = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Sheet-Metal3", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swSheetMetalFeatureData = (SheetMetalFeatureData)swFeature.GetDefinition();
            Debug.Print("Get override parameter values for Sheet-Metal3:");
            errors = swSheetMetalFeatureData.GetOverrideDefaultParameter2((int)swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_BendParameters, out overridden);
            Debug.Print(" Bend parameters: " + overridden);
            errors = swSheetMetalFeatureData.GetOverrideDefaultParameter2((int)swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_BendAllowance, out overridden);
            Debug.Print(" Bend allowance: " + overridden);
            errors = swSheetMetalFeatureData.GetOverrideDefaultParameter2((int)swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRelief, out overridden);
            Debug.Print(" Auto relief: " + overridden);
            swSheetMetalFeatureData.AccessSelections(swModel, null);
            if (overridden)
            {
                errors = swSheetMetalFeatureData.SetOverrideDefaultParameter2((int)swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRelief, false);
            }
            else
            {
                errors = swSheetMetalFeatureData.SetOverrideDefaultParameter2((int)swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRelief, true);
            }
            swFeature.ModifyDefinition(swSheetMetalFeatureData, swModel, null);
            errors = swSheetMetalFeatureData.GetOverrideDefaultParameter2((int)swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRelief, out overridden);
            Debug.Print("Override auto relief default parameters for Sheet-Metal3? " + overridden);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Set Override Option for Auto Relief Default Parameters Example (VBA)

This example shows how to set the override option for the auto relief default parameters of a sheet metal feature in a multibody sheet metal part.

```
'-----
' Preconditions:
' 1. Open a multibody sheet metal part that contains
'    a Sheet-Metal3 feature in the Sheet-Metal folder.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets whether the default parameters are overridden
'    for Sheet-Metal3.
' 2. Sets the override option for the auto relief default
'    parameters of Sheet-Metal3.
' 3. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swSheetMetalFeatureData As SldWorks.SheetMetalFeatureData
Dim errors As Long
Dim status As Boolean
Dim override As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("Sheet-Metal3", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swSheetMetalFeatureData = swFeature.GetDefinition
    Debug.Print ("Get override parameter values for Sheet-Metal3:")
    errors =
swSheetMetalFeatureData.GetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_BendParameters,
override)
    Debug.Print (" Bend parameters: " & override)
    errors =
swSheetMetalFeatureData.GetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_BendAllowance,
override)
    Debug.Print (" Bend allowance: " & override)
    errors =
swSheetMetalFeatureData.GetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRelief, override)
    Debug.Print (" Auto relief: " & override)
    swSheetMetalFeatureData.AccessSelections swModel, Nothing
    If override Then
        errors =
swSheetMetalFeatureData.SetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRelief, False)
    Else
        errors =
swSheetMetalFeatureData.SetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRelief, True)
    End If
    swFeature.ModifyDefinition swSheetMetalFeatureData, swModel, Nothing
    errors =
swSheetMetalFeatureData.GetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRelief, override)
    Debug.Print ("Override auto relief default parameters for Sheet-Metal3? " & override)

End Sub
```

Set Override Option for Auto Relief Default Parameters Example (VB.NET)

This example shows how to set the override option for the auto relief default parameters of a sheet metal feature in a multibody sheet metal part.

```

'-----
' Preconditions:
' 1. Open a multibody sheet metal part that contains
'    a Sheet-Metal3 feature in the Sheet-Metal folder.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets whether the default parameters are overridden
'    for Sheet-Metal3.
' 2. Sets the override option for the auto relief default
'    parameters of Sheet-Metal3.
' 3. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
'       save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swSheetMetalFeatureData As SheetMetalFeatureData
        Dim errors As Integer
        Dim status As Boolean
        Dim override As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("Sheet-Metal3", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swSheetMetalFeatureData = swFeature.GetDefinition
        Debug.Print("Get override parameter values for Sheet-Metal3:")
        errors = swSheetMetalFeatureData.GetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_BendPa
rameters, override)
        Debug.Print("  Bend parameters: " & override)
        errors = swSheetMetalFeatureData.GetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_BendAl
lowance, override)
        Debug.Print("  Bend allowance: " & override)
        errors = swSheetMetalFeatureData.GetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRe
lief, override)
        Debug.Print("  Auto relief: " & override)
        swSheetMetalFeatureData.AccessSelections(swModel, Nothing)
        If override Then
            errors = swSheetMetalFeatureData.SetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_Au
toRelief, False)
        Else
            errors = swSheetMetalFeatureData.SetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_Au
toRelief, True)
        End If
        swFeature.ModifyDefinition(swSheetMetalFeatureData, swModel, Nothing)
        errors = swSheetMetalFeatureData.GetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRe
lief, override)
        Debug.Print("Override auto relief default parameters for Sheet-Metal3? " & override)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Set Profile for Structural Member Example (VBA)

This example shows how to set the profile for a structural member.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
' 2. Verify that the path specified for the weldment profile
'    exists.
' 3. Right-click Structural Member1, select Edit Feature,
'    examine Selections, and click OK.
'
' Postconditions:
' 1. Changes the weldment profile to the specified weldment
'    profile.
' 2. To verify step 1, right-click StructuralMember1, select
'    Edit Feature, examine Selections, and click OK.
'
' NOTE: Because the part is used elsewhere, do not save
' changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swWeldFeat As SldWorks.Feature
Dim swWeldFeatData As SldWorks.StructuralMemberFeatureData
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SelectByID("Structural Member1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swWeldFeat = swSelMgr.GetSelectedObject6(1, 0)
    Set swWeldFeatData = swWeldFeat.GetDefinition
    swWeldFeatData.AccessSelections swModel, Nothing
    swWeldFeatData.WeldmentProfilePath = "C:\Program Files\SolidWorks Corp\SolidWorks\lang\english\weldment profiles\iso\pipe\26.9 x 3.2.sldlfp"
    boolstatus = swWeldFeat.ModifyDefinition(swWeldFeatData, swModel, Nothing)

End Sub
```

Insert Angular Running Dimension Example (C#)

This example shows how to insert an angular running dimension and get its properties.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. The specified angular running dimension is inserted into the drawing.
// 2. Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes to it.
// -----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace AngularRunningDimensions_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = default(ModelDoc2);
            SelectionMgr selmgr = default(SelectionMgr);
            DisplayDimension dispdim = default(DisplayDimension);
            object dispdimvar;
            bool boolstatus = false;
            int intstatus;

            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByID("", "EDGE", 0.163726736787323, 0.199115091463415, 0.00479999999993197, true, 0,
null, 0);
            boolstatus = Part.Extension.SelectByID("", "EDGE", 0.220795425811714, 0.179644597560976, 0.00479999999998881, true, 0,
null, 0);

            dispdimvar = Part.Extension.AddAngularRunningDim(false, true, true, 0.154288188900673, 0.0794194886913027, 0, out
intstatus);
            Part.Extension.ReJogRunningDimension();
            Part.Extension.AlignRunningDimension();

            Part.SetPickMode();

            boolstatus = Part.Extension.SelectByID("D2@Sketch31@foodprocessor.SLDDRW", "DIMENSION", 0.0408612062995185,
0.166216670731707, 0, false, 0, null, 0);
            selmgr = (SelectionMgr)Part.SelectionManager;
            dispdim = (DisplayDimension)selmgr.GetSelectedObject6(1, -1);

            Debug.Print("Display chained angular dimensions? " + dispdim.DisplayAsChain);
            Debug.Print("Run the angular dimensions bidirectionally? " + dispdim.RunBidirectionally);
            Debug.Print("Extend extension lines from center of angular running dimension? " +
dispdim.ExtensionLineExtendsFromCenterOfSet);
            Debug.Print("Are extension lines jogged? " + dispdim.Jogged);
            Debug.Print("Extension line style same as leader line style? " + dispdim.ExtensionLineSameAsLeaderStyle);
            Debug.Print("Extension line uses document settings? " + dispdim.ExtensionLineUseDocumentDisplay);

        }

        public SldWorks swApp;
    }
}
```

Insert Angular Running Dimension Example (VBA)

This example shows how to insert an angular running dimension and get its properties.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. The specified angular running dimension is inserted into the drawing.
' 2. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes to it.
'-----
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim selmgr As SldWorks.SelectionMgr
Dim dispdimvar As Variant
Dim dispdim As SldWorks.DisplayDimension
Dim boolstatus As Boolean
Dim errstatus As Long
Option Explicit
Sub main()
    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc

    boolstatus = Part.ActivateView("Drawing View1")
    boolstatus = Part.Extension.SelectByID2("", "EDGE", 0.163726736787323, 0.199115091463415, 4.79999999993197E-03, True, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("", "EDGE", 0.220795425811714, 0.179644597560976, 4.79999999998881E-03, True, 0, Nothing, 0)

    Set dispdimvar = Part.Extension.AddAngularRunningDim(False, True, True, 0.154288188900673, 7.94194886913027E-02, 0, errstatus)
    Part.Extension.ReJogRunningDimension
    Part.Extension.AlignRunningDimension

    Part.SetPickMode

    boolstatus = Part.Extension.SelectByID2("D2@Sketch31@foodprocessor.SLDDRW", "DIMENSION", 4.08612062995185E-02, 0.166216670731707, 0, False, 0, Nothing, 0)
    Set selmgr = Part.SelectionManager
    Set dispdim = selmgr.GetSelectedObject6(1, -1)

    Debug.Print "Display chained angular dimensions? " & dispdim.DisplayAsChain
    Debug.Print "Run the angular dimensions bidirectionally? " & dispdim.RunBidirectionally
    Debug.Print "Extend extension lines from center of angular running dimension? " & dispdim.ExtensionLineExtendsFromCenterOfSet
    Debug.Print "Are extension lines jogged? " & dispdim.Jogged
    Debug.Print "Extension line style same as leader line style? " & dispdim.ExtensionLineSameAsLeaderStyle
    Debug.Print "Extension line uses document settings? " & dispdim.ExtensionLineUseDocumentDisplay

End Sub

```

Insert Angular Running Dimension Example (VB.NET)

This example shows how to insert an angular running dimension and get its properties.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. The specified angular running dimension is inserted into the drawing.
' 2. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes to it.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim Part As ModelDoc2
        Dim selmgr As SelectionMgr
        Dim dispdim As DisplayDimension
        Dim dispdimvar As Object
        Dim boolstatus As Boolean
        Dim errstatus As Integer

```

```
        Part = swApp.ActiveDoc
```

```

        boolstatus = Part.ActivateView("Drawing View1")
        boolstatus = Part.Extension.SelectByID2("", "EDGE", 0.163726736787323, 0.199115091463415, 0.00479999999993197, True, 0,
Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("", "EDGE", 0.220795425811714, 0.179644597560976, 0.00479999999998881, True, 0,
Nothing, 0)

```

```

        dispdimvar = Part.Extension.AddAngularRunningDim(False, True, True, 0.154288188900673, 0.0794194886913027, 0, errstatus)
        Part.Extension.ReJogRunningDimension()
        Part.Extension.AlignRunningDimension()

```

```
        Part.SetPickMode()
```

```

        boolstatus = Part.Extension.SelectByID2("D2@Sketch31@foodprocessor.SLDDRW", "DIMENSION", 0.0408612062995185,
0.166216670731707, 0, False, 0, Nothing, 0)
        selmgr = Part.SelectionManager
        dispdim = selmgr.GetSelectedObject6(1, -1)

```

```

        Debug.Print("Display chained angular dimensions? " & dispdim.DisplayAsChain)
        Debug.Print("Run the angular dimensions bidirectionally? " & dispdim.RunBidirectionally)
        Debug.Print("Extend extension lines from center of angular running dimension? " &
dispdim.ExtensionLineExtendsFromCenterOfSet)
        Debug.Print("Are extension lines jogged? " & dispdim.Jogged)
        Debug.Print("Extension line style same as leader line style? " & dispdim.ExtensionLineSameAsLeaderStyle)
        Debug.Print("Extension line uses document settings? " & dispdim.ExtensionLineUseDocumentDisplay)

```

```
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Set Rounding of Decimal Units in Display Dimensions (C#)

This example shows how to specify the rounding of decimal units in display dimensions.

```
//-----
// Preconditions: Open:
//   public_documents\samples\tutorial\api\box.slddrw
//
// Postconditions:
// 1. Click the display dimension in the graphics area.
// 2. Under Dual Dimension on the Value tab, observe that Inward rounding
//    is selected.
// 3. Under Override Units on the Other tab, observe that:
//    * Decimal is clicked.
//    * Feet & Inches is selected.
//    * Truncate without rounding is selected.
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace SetDual2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        SelectionMgr swSelMgr;
        DisplayDimension swDispDim;
        bool boolstatus;

        int longstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            swSelMgr = (SelectionMgr)Part.SelectionManager;
            boolstatus = Part.Extension.SelectByID2("RD1@Drawing View1", "DIMENSION", 0.305961854224123, 0.247549077953509, 0,
false, 0, null, 0);
            swDispDim = (DisplayDimension)swSelMgr.GetSelectedObject6(1, 0);

            longstatus = swDispDim.SetUnits2(false, (int)swLengthUnit_e.swFEETINCHES, (int)swFractionDisplay_e.swDECIMAL, 0, false,
(int)swUnitsDecimalRounding_e.swUnitsDecimalRounding_Truncate);
            swDispDim.SetDual2(false, true);

        }

        public SldWorks swApp;
    }
}
```

Set Rounding of Decimal Units in Display Dimensions (VBA)

This example shows how to specify the rounding of decimal units in display dimensions.

```

'-----
' Preconditions: Open:
'   public_documents\samples\tutorial\api\box.slddrw
'
' Postconditions:
' 1. Click the display dimension in the graphics area.
' 2. Under Dual Dimension on the Value tab, observe that Inward rounding
'    is selected.
' 3. Under Override Units on the Other tab, observe that:
'    * Decimal is clicked.
'    * Feet & Inches is selected.
'    * Truncate without rounding is selected.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

Dim swApp                As SldWorks.SldWorks
Dim Part                 As SldWorks.ModelDoc2
Dim swSelMgr             As SldWorks.SelectionMgr
Dim swDispDim            As SldWorks.DisplayDimension
Dim boolstatus           As Boolean
Dim longstatus           As Long
Option Explicit
Sub main()
    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc

    Set swSelMgr = Part.SelectionManager
    boolstatus = Part.Extension.SelectByID2("RD1@Drawing View1", "DIMENSION", 0.305961854224123, 0.247549077953509, 0, False, 0,
Nothing, 0)
    Set swDispDim = swSelMgr.GetSelectedObject6(1, 0)

    longstatus = swDispDim.SetUnits2(False, swLengthUnit_e.swFEETINCHES, swFractionDisplay_e.swDECIMAL, 0, False,
swUnitsDecimalRounding_e.swUnitsDecimalRounding_Truncate)
    Call swDispDim.SetDual2(False, True)
End Sub

```

Set Rounding of Decimal Units in Display Dimensions (VB.NET)

This example shows how to specify the rounding of decimal units in display dimensions.

```

' Preconditions: Open:
'   public_documents\samples\tutorial\api\box.slddrw
'
' Postconditions:
' 1. Click the display dimension in the graphics area.
' 2. Under Dual Dimension on the Value tab, observe that Inward rounding
'    is selected.
' 3. Under Override Units on the Other tab, observe that:
'    * Decimal is clicked.
'    * Feet & Inches is selected.
'    * Truncate without rounding is selected.
'
' NOTE: Because the model is used elsewhere, do not save changes.

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
```

```
    Dim swSelMgr As SelectionMgr
```

```
    Dim swDispDim As DisplayDimension
```

```
    Dim boolstatus As Boolean
```

```
    Dim longstatus As Integer
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc
```

```
        swSelMgr = Part.SelectionManager
```

```
        boolstatus = Part.Extension.SelectByID2("RD1@Drawing View1", "DIMENSION", 0.305961854224123, 0.247549077953509, 0, False,
0, Nothing, 0)
```

```
        swDispDim = swSelMgr.GetSelectedObject6(1, 0)
```

```
        longstatus = swDispDim.SetUnits2(False, swLengthUnit_e.swFEETINCHES, swFractionDisplay_e.swDECIMAL, 0, False,
swUnitsDecimalRounding_e.swUnitsDecimalRounding_Truncate)
```

```
        Call swDispDim.SetDual2(False, True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Set Spline Tangency and Curvature Controls Example (VBA)

This example show how to set a spline's tangency and curvature controls.

```
'-----  
' Preconditions:  
' 1. Open a part containing a sketch of a spline.  
' 2. Select the spline in the graphics area.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Sets the tangency and curvature control  
'    for the specified points.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSpline As SldWorks.SketchSpline  
Dim swSplineH1 As SldWorks.SplineHandle  
Dim swSplineH2 As SldWorks.SplineHandle  
Dim boolstatus As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swModelDocExt = swModel.Extension  
    boolstatus = swModelDocExt.SelectByID2("Spline1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)  
    Set swSpline = swSelMgr.GetSelectedObject6(1, 0)  
  
    ' Set tangency control; substitute coordinates of a point in your spline  
    Set swSplineH1 = swSpline.AddTangencyControl(-48.92 / 1000, -5.07 / 1000, 0)  
    Debug.Print "Spline point number: " & swSplineH1.SplinePointNumber()  
  
    ' Add curvature control; substitute coordinates of a point in your spline  
    Set swSplineH2 = swSpline.AddCurvatureControl(6.3 / 1000, 15.075 / 1000, 0)  
    Debug.Print "Spline point number: " & swSplineH2.SplinePointNumber()  
  
    ' Reset all handles  
    swSpline.ResetAllHandles  
    ' Relax spline  
    swSpline.RelaxSpline  
  
    swModel.WindowRedraw  
  
End Sub
```

Set Options for Parting Line Example (VBA)

This example shows how to set the split faces and core/cavity split options for a parting line.

```
'-----
' Preconditions:
' 1. Open a part document containing a parting line
'    feature.
' 2. Select the parting line feature.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Sets the split faces and core/cavity split
'    options.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swPartingLine As SldWorks.PartingLineFeatureData
    Dim i As Long
    Dim bRet As Boolean
    Dim SplitFaceOption As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, 0)
    Set swPartingLine = swFeat.GetDefinition

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swFeat.Name
    Debug.Print " " & swPartingLine.Angle
    Debug.Print " Split faces = " & swPartingLine.SplitFaces
    Debug.Print " Core/cavity split = " & swPartingLine.CoreCavitySplit

    SplitFaceOption = swPartingLine.SplitFacesOption
    swPartingLine.SplitFacesOption = swSplitFacesAtPlusMinusDraftTransition
    Debug.Print " Split faces option = " & swPartingLine.SplitFacesOption

    bRet = swPartingLine.AccessSelections(swModel, Nothing): Debug.Assert bRet

    swPartingLine.SplitFaces = True
    swPartingLine.CoreCavitySplit = True

    Debug.Print " "
    Debug.Print " Split faces = " & swPartingLine.SplitFaces
    Debug.Print " Core/cavity split = " & swPartingLine.CoreCavitySplit
    swPartingLine.SplitFacesOption = swSplitFacesAtSpecifiedAngle
    Debug.Print " Split faces option = " & swPartingLine.SplitFacesOption

    bRet = swFeat.ModifyDefinition(swPartingLine, swModel, Nothing): Debug.Assert bRet

End Sub
```

Set Table Anchors in Sheet Formats Example (C#)

This example shows how to set a hole table anchor in a sheet format of a drawing.

```
//-----
// Preconditions:
// 1. Verify that the specified drawing to open exists.
// 2. Open and Immediate window.
//
// Postconditions:
// 1. Opens the drawing.
// 2. At Stop, examine the position of the hole table in the drawing.
// 3. Press F5.
// 4. Edits the sheet format.
// 5. Creates a point.
// 6. Sets the position of the hole table's anchor at the new point.
// 7. Examine the new position of the hole table
//    and the Immediate window.
//
// NOTE: If prompted, do not save changes when closing the drawing.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;

namespace SetTableAnchor_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            DrawingDoc swDrw = default(DrawingDoc);
            Sheet swSheet = default(Sheet);
            ModelDoc2 swModel = default(ModelDoc2);
            SketchPoint swSkPoint = default(SketchPoint);
            TableAnchor oldTableAnchor = default(TableAnchor);
            TableAnchor newTableAnchor = default(TableAnchor);
            double[] vPosition = null;
            string filename = null;
            int errors = 0;
            int warnings = 0;

            filename = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\SimpleHole.slddrw";
            swDrw = (DrawingDoc)swApp.OpenDoc6(filename, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptic

            if (swDrw == null)
                return;

            System.Diagnostics.Debugger.Break();

            swSheet = (Sheet)swDrw.GetCurrentSheet();
            swModel = (ModelDoc2)swDrw;

            swModel.Extension.SelectByID2(swSheet.GetName(), "SHEET", 0, 0, 0, false, 0, null, 0);

            swDrw.EditTemplate();
            swModel.EditSketch();

            swSkPoint = swModel.SketchManager.CreatePoint(0.2, 0.07, 0);
            swSkPoint.Select4(false, null);

            // If an anchor for the hole table already exists, move it to the selected sketch point
            oldTableAnchor = swSheet.get_TableAnchor((int)swTableAnnotationType_e.swTableAnnotation_HoleChart);
            newTableAnchor = (TableAnchor)swSheet.SetAsTableAnchor((int)swTableAnnotationType_e.swTableAnnotation_HoleChart);

            if ((newTableAnchor != null))
            {
                vPosition = (double[])newTableAnchor.Position;
                Debug.Print("Table Anchor at (" + vPosition[0] + ", " + vPosition[1] + ")");
            }

            swDrw.EditSheet();
            swModel.EditSketch();
            swModel.ForceRebuild3(true);
        }

        // The SldWorks swApp variable is pre-assigned for you.
        public SldWorks swApp;
    }
}
```

Set Table Anchors in Sheet Formats Example (VBA)

This example shows how to set a hole table anchor in a sheet format of a drawing.

```

'-----
' Preconditions:
' 1. Verify that the specified drawing to open exists.
' 2. Open and Immediate window.
'
' Postconditions:
' 1. Opens the drawing.
' 2. At Stop, examine the position of the hole table in the drawing.
' 3. Press F5.
' 4. Edits the sheet format.
' 5. Creates a point.
' 6. Sets the position of the hole table's anchor at the new point.
' 7. Examine the new position of the hole table
'    and the Immediate window.
'
' NOTE: If prompted, do not save changes when closing the drawing.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swDrw As DrawingDoc
Dim swSheet As Sheet
Dim swModel As ModelDoc2
Dim swSkPoint As SketchPoint
Dim oldTableAnchor As TableAnchor
Dim newTableAnchor As TableAnchor
Dim vPosition As Variant
Dim filename As String
Dim errors As Long
Dim warnings As Long

```

Sub main()

```

    Set swApp = Application.SldWorks
    filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\SimpleHole.slddrw"
    Set swDrw = swApp.OpenDoc6(filename, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)

```

```

    If swDrw Is Nothing Then Exit Sub

```

Stop

```

    Set swSheet = swDrw.GetCurrentSheet()
    Set swModel = swDrw

```

```

    swModel.Extension.SelectByID2 swSheet.GetName(), "SHEET", 0, 0, 0, False, 0, Nothing, 0

```

```

    swDrw.EditTemplate
    swModel.EditSketch

```

```

    Set swSkPoint = swModel.SketchManager.CreatePoint(0.2, 0.07, 0)
    swSkPoint.Select4 False, Nothing

```

```

    ' If an anchor for the hole table already exists, move it to the selected sketch point
    Set oldTableAnchor = swSheet.TableAnchor(swTableAnnotation_HoleChart)
    Set newTableAnchor = swSheet.SetAsTableAnchor(swTableAnnotation_HoleChart)

```

```

    If Not newTableAnchor Is Nothing Then

```

```

        vPosition = newTableAnchor.Position
        Debug.Print "Table Anchor at (" & vPosition(0) & ", " & vPosition(1) & ")"

```

```

    End If

```

```

    swDrw.EditSheet
    swModel.EditSketch
    swModel.ForceRebuild3 True

```

End Sub

Set Table Anchors in Sheet Formats Example (VB.NET)

This example shows how to set a hole table anchor in a sheet format of a drawing.

```

'-----
' Preconditions:
' 1. Verify that the specified drawing to open exists.
' 2. Open and Immediate window.
'
' Postconditions:
' 1. Opens the drawing.
' 2. At Stop, examine the position of the hole table in the drawing.
' 3. Press F5.
' 4. Edits the sheet format.
' 5. Creates a point.
' 6. Sets the position of the hole table's anchor at the new point.
' 7. Examine the new position of the hole table
'    and the Immediate window.
'
' NOTE: If prompted, do not save changes when closing the drawing.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```

        Dim swDrw As DrawingDoc
        Dim swSheet As Sheet
        Dim swModel As ModelDoc2
        Dim swSkPoint As SketchPoint
        Dim oldTableAnchor As TableAnchor
        Dim newTableAnchor As TableAnchor
        Dim vPosition As Object
        Dim filename As String
        Dim errors As Integer
        Dim warnings As Integer
    
```

```

        filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\SimpleHole.slddrw"
        swDrw = swApp.OpenDoc6(filename, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, war
    
```

```
        If swDrw Is Nothing Then Exit Sub
```

```
        Stop
```

```

        swSheet = swDrw.GetCurrentSheet()
        swModel = swDrw
    
```

```
        swModel.Extension.SelectByID2(swSheet.GetName(), "SHEET", 0, 0, 0, False, 0, Nothing, 0)
```

```

        swDrw.EditTemplate
        swModel.EditSketch
    
```

```

        swSkPoint = swModel.SketchManager.CreatePoint(0.2, 0.07, 0)
        swSkPoint.Select4(False, Nothing)
    
```

```

' If an anchor for the hole table already exists, move it to the selected sketch point
oldTableAnchor = swSheet.TableAnchor(swTableAnnotationType_e.swTableAnnotation_HoleChart)
newTableAnchor = swSheet.SetAsTableAnchor(swTableAnnotationType_e.swTableAnnotation_HoleChart)
    
```

```
    If Not newTableAnchor Is Nothing Then
```

```

        vPosition = newTableAnchor.Position
        Debug.Print("Table Anchor at (" & vPosition(0) & ", " & vPosition(1) & ")")
    
```

```
    End If
```

```

        swDrw.EditSheet
        swModel.EditSketch
        swModel.ForceRebuild3(True)
    
```

```
End Sub
```

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
    
```

```
End Class
```

Set Text in Datum Tags and GTols Example (C#)

This example shows how to set the text in datum tags and geometric tolerances.

```
//-----
// Preconditions:
// 1. Open a drawing that contains at least one view, a datum feature,
//    and a geometric tolerance.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Sets prefix, suffix, and callouts for the first GTol.
// 2. Inserts below frame text in the first GTol.
// 3. Edits below frame text.
// 4. Deletes the first line of the below frame.
// 5. Gets the datum tag label.
// 6. Examine the Immediate window.
//-----

using System;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace InsertEditBelowFrame_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelView swModelView = default(ModelView);
            DrawingDoc swDraw = default(DrawingDoc);
            View swView = default(View);
            GTol swDispGTol = default(GTol);
            DatumTag swDatumTag = default(DatumTag);
            double Rect = 0;
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelView = (ModelView)swModel.ActiveView;
            swDraw = (DrawingDoc)swModel;
            swView = (View)swDraw.GetFirstView();
            //sheet
            swView = (View)swView.GetNextView();

            //Set text parts
            swDispGTol = (GTol)swView.GetFirstGTOL();
            swDispGTol.SetText((int)swGTolTextParts_e.swGTolTextPrefix, "Prefix");
            swDispGTol.SetText((int)swGTolTextParts_e.swGTolTextSuffix, "Suffix");
            swDispGTol.SetText((int)swGTolTextParts_e.swGTolTextCalloutAbove, "Above");
            swDispGTol.SetText((int)swGTolTextParts_e.swGTolTextCalloutBelow, "Below");
            swDispGTol.SetText((int)swGTolTextParts_e.swGTolTextCalloutBelow, "Below1");
            swDispGTol.SetText((int)swGTolTextParts_e.swGTolTextCalloutBelow, "Below2");

            swModelView.GraphicsRedraw(Rect);

            //Get text parts
            Debug.Print("Callout above: " + swDispGTol.GetText((int)swGTolTextParts_e.swGTolTextCalloutAbove));
            Debug.Print("Callout below: " + swDispGTol.GetText((int)swGTolTextParts_e.swGTolTextCalloutBelow));
            Debug.Print("Prefix: " + swDispGTol.GetText((int)swGTolTextParts_e.swGTolTextPrefix));
            Debug.Print("Suffix: " + swDispGTol.GetText((int)swGTolTextParts_e.swGTolTextSuffix));

            //Insert below frame text
            bRet = swDispGTol.InsertBelowFrameTextAt(1, "Line 1 of below frame");
            bRet = swDispGTol.InsertBelowFrameTextAt(2, "Line 2 of below frame");
            bRet = swDispGTol.InsertBelowFrameTextAt(3, "Line 3 of below frame");

            //Get below frame text
            Debug.Print("Below frame text:");
            Debug.Print("  swDispGTol.GetBelowFrameTextAt(1)");
            Debug.Print("  swDispGTol.GetBelowFrameTextAt(2)");
            Debug.Print("  swDispGTol.GetBelowFrameTextAt(3)");

            //Inserts a line at index position 2, because overwrite is set to False
            bRet = swDispGTol.SetBelowFrameTextAt(2, "Line 2 edited in below frame", false);
            Debug.Print("  swDispGTol.GetBelowFrameTextAt(2)");

            bRet = swDispGTol.DeleteBelowFrameTextAt(1);
            //Deletes the first line in the below frame
            //bRet = swDispGTol.DeleteBelowFrameTextAt(-1) 'Deletes all below frame text

            swDatumTag = (DatumTag)swView.GetFirstDatumTag();
            swDatumTag.SetText((int)swDatumTagTextParts_e.swDatumTagTextPrefix, "prefix");
            swDatumTag.SetText((int)swDatumTagTextParts_e.swDatumTagTextSuffix, "suffix");
            swDatumTag.SetText((int)swDatumTagTextParts_e.swDatumTagTextCalloutAbove, "above");
        }
    }
}
```

```
swDatumTag.SetText((int)swDatumTagTextParts_e.swDatumTagTextCalloutBelow, "below");
Debug.Print("Datum tag label: " + swDatumTag.GetLabel());

swModelView.GraphicsRedraw(Rect);
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;
}
}
```

Set Text in Datum Tags and GTols Example (VBA)

This example shows how to set the text in datum tags and geometric tolerances.

```

'-----
' Preconditions:
' 1. Open a drawing that contains at least one view, a datum feature,
'    and a geometric tolerance.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Sets prefix, suffix, and callouts for the first GTol.
' 2. Inserts below frame text in the first GTol.
' 3. Edits below frame text.
' 4. Deletes the first line of the below frame.
' 5. Gets the datum tag label.
' 6. Examine the Immediate window.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelView As SldWorks.ModelView
Dim swDraw As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim swDispGtol As SldWorks.Gtol
Dim swDatumTag As SldWorks.DatumTag
Dim Rect As Variant

```

```
Dim bRet as Boolean
```

Sub main()

```

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swModelView = swModel.ActiveView
Set swDraw = swModel
Set swView = swDraw.GetFirstView 'sheet
Set swView = swView.GetNextView

```

```

'Set text parts
Set swDispGtol = swView.GetFirstGTOL
swDispGtol.SetText swGTolTextPrefix, "Prefix"
swDispGtol.SetText swGTolTextSuffix, "Suffix"
swDispGtol.SetText swGTolTextCalloutAbove, "Above"
swDispGtol.SetText swGTolTextCalloutBelow, "Below"
swDispGtol.SetText swGTolTextCalloutBelow, "Below1"
swDispGtol.SetText swGTolTextCalloutBelow, "Below2"

```

```
swModelView.GraphicsRedraw (Rect)
```

```

'Get text parts
Debug.Print "Callout above: " & swDispGtol.GetText(swGTolTextParts_e.swGTolTextCalloutAbove)
Debug.Print "Callout below: " & swDispGtol.GetText(swGTolTextParts_e.swGTolTextCalloutBelow)
Debug.Print "Prefix: " & swDispGtol.GetText(swGTolTextParts_e.swGTolTextPrefix)
Debug.Print "Suffix: " & swDispGtol.GetText(swGTolTextParts_e.swGTolTextSuffix)

```

```

'Insert below frame text
bRet = swDispGtol.InsertBelowFrameTextAt(1, "Line 1 of below frame")
bRet = swDispGtol.InsertBelowFrameTextAt(2, "Line 2 of below frame")
bRet = swDispGtol.InsertBelowFrameTextAt(3, "Line 3 of below frame")

```

```

'Get below frame text
Debug.Print "Below frame text:"
Debug.Print "  swDispGtol.GetBelowFrameTextAt(1)"
Debug.Print "  swDispGtol.GetBelowFrameTextAt(2)"
Debug.Print "  swDispGtol.GetBelowFrameTextAt(3)"

```

```

'Inserts a line at index position 2, because overwrite is set to False
bRet = swDispGtol.SetBelowFrameTextAt(2, "Line 2 edited in below frame", False)
Debug.Print "  swDispGtol.GetBelowFrameTextAt(2)"

```

```

bRet = swDispGtol.DeleteBelowFrameTextAt(1) 'Deletes the first line in the below frame
bRet = swDispGtol.DeleteBelowFrameTextAt(-1) 'Deletes all below frame text

```

```

Set swDatumTag = swView.GetFirstDatumTag
swDatumTag.SetText swDatumTagTextPrefix, "prefix"
swDatumTag.SetText swDatumTagTextSuffix, "suffix"
swDatumTag.SetText swDatumTagTextCalloutAbove, "above"
swDatumTag.SetText swDatumTagTextCalloutBelow, "below"
Debug.Print ("Datum tag label: " & swDatumTag.GetLabel)

```

```

Set Rect = Nothing
swModelView.GraphicsRedraw (Rect)

```

End Sub

Set Text in Datum Tags and GTols Example (VB.NET)

This example shows how to set the text in datum tags and geometric tolerances.

```

'-----
' Preconditions:
' 1. Open a drawing that contains at least one view, a datum feature,
'    and a geometric tolerance.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Sets prefix, suffix, and callouts for the first GTol.
' 2. Inserts below frame text in the first GTol.
' 3. Edits below frame text.
' 4. Deletes the first line of the below frame.
' 5. Gets the datum tag label.
' 6. Examine the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
Public Sub main()
```

```

    Dim swModel As ModelDoc2
    Dim swModelView As ModelView
    Dim swDraw As DrawingDoc
    Dim swView As View
    Dim swDispGtol As Gtol
    Dim swDatumTag As DatumTag
    Dim Rect As Double
    Dim bRet As Boolean

    swModel = swApp.ActiveDoc
    swModelView = swModel.ActiveView
    swDraw = swModel
    swView = swDraw.GetFirstView 'sheet
    swView = swView.GetNextView

    'Set text parts
    swDispGtol = swView.GetFirstGTOL
    swDispGtol.SetText(swGTolTextParts_e.swGTolTextPrefix, "Prefix")
    swDispGtol.SetText(swGTolTextParts_e.swGTolTextSuffix, "Suffix")
    swDispGtol.SetText(swGTolTextParts_e.swGTolTextCalloutAbove, "Above")
    swDispGtol.SetText(swGTolTextParts_e.swGTolTextCalloutBelow, "Below")
    swDispGtol.SetText(swGTolTextParts_e.swGTolTextCalloutBelow, "Below1")
    swDispGtol.SetText(swGTolTextParts_e.swGTolTextCalloutBelow, "Below2")

    swModelView.GraphicsRedraw(Rect)

    'Get text parts
    Debug.Print("Callout above: " & swDispGtol.GetText(swGTolTextParts_e.swGTolTextCalloutAbove))
    Debug.Print("Callout below: " & swDispGtol.GetText(swGTolTextParts_e.swGTolTextCalloutBelow))
    Debug.Print("Prefix: " & swDispGtol.GetText(swGTolTextParts_e.swGTolTextPrefix))
    Debug.Print("Suffix: " & swDispGtol.GetText(swGTolTextParts_e.swGTolTextSuffix))

    'Insert below frame text
    bRet = swDispGtol.InsertBelowFrameTextAt(1, "Line 1 of below frame")
    bRet = swDispGtol.InsertBelowFrameTextAt(2, "Line 2 of below frame")
    bRet = swDispGtol.InsertBelowFrameTextAt(3, "Line 3 of below frame")

    'Get below frame text
    Debug.Print("Below frame text:")
    Debug.Print("  swDispGtol.GetBelowFrameTextAt(1)")
    Debug.Print("  swDispGtol.GetBelowFrameTextAt(2)")
    Debug.Print("  swDispGtol.GetBelowFrameTextAt(3)")

    'Inserts a line at index position 2, because overwrite is set to False
    bRet = swDispGtol.SetBelowFrameTextAt(2, "Line 2 edited in below frame", False)
    Debug.Print("  swDispGtol.GetBelowFrameTextAt(2)")

    bRet = swDispGtol.DeleteBelowFrameTextAt(1) 'Deletes the first line in the below frame
    'bRet = swDispGtol.DeleteBelowFrameTextAt(-1) 'Deletes all below frame text

    swDatumTag = swView.GetFirstDatumTag
    swDatumTag.SetText(swDatumTagTextParts_e.swDatumTagTextPrefix, "prefix")
    swDatumTag.SetText(swDatumTagTextParts_e.swDatumTagTextSuffix, "suffix")
    swDatumTag.SetText(swDatumTagTextParts_e.swDatumTagTextCalloutAbove, "above")
    swDatumTag.SetText(swDatumTagTextParts_e.swDatumTagTextCalloutBelow, "below")
    Debug.Print("Datum tag label: " & swDatumTag.GetLabel)

    Rect = Nothing

```

```
swModelView.GraphicsRedraw(Rect)
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Set Transparency of Unmodified Components in Large Design Review Assembly Example (C#)

This example shows how to:

- open an assembly in Large Design Review mode
- open, modify, save, and close an assembly component, and
- set the transparency level of unmodified components in the assembly.

```
//-----
// Preconditions:
// 1. Create c:\temp\LDR.
// 2. Copy these files from public_documents\samples\tutorial\api to c:\temp\LDR:
//    * landing_gear.sldasm
//    * lwrsway_lnk.sldprt
//    * oleopiston.sldprt
//    * oleostrut.sldprt
//    * part4.sldprt
//    * upprsway_lnk.sldprt
//    * wheel_hub.sldprt
//    * wheelassy.sldasm
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Click OK in the Large Design Review dialog.
// 2. Opens the assembly in Large Design Review mode.
// 3. Opens, modifies, saves, and closes the assembly component, oleostrut.sldprt.
//    a. When prompted to rebuild, click Rebuild and save the document.
//    b. If prompted to update the graphics data in Large Design Review,
//       click Yes.
// 4. Sets the transparency level of unmodified components in the assembly to 0.75.
// 5. Close the assembly and part documents.
// 6. Examine the Immediate window and graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace LargeDesignReviewCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            AssemblyDoc swAssemblyDoc = default(AssemblyDoc);
            Dimension swDimension = default(Dimension);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            // Open an assembly in Large Design Review mode
            fileName = "C:\\temp\\LDR\\landing_gear.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_ViewOnly, "", ref
errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swAssemblyDoc = (AssemblyDoc)swApp.ActiveDoc;

            // Open a component, modify it, and save it
            ModelDoc2 swCompModel = default(ModelDoc2);
            status = swModelDocExt.SelectByID2("oleostrut-1@landing_gear", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swApp.OpenDoc6("C:\\temp\\LDR\\oleostrut.sldprt", 1, 0, "", ref errors, ref warnings);
            swApp.ActivateDoc3("oleostrut.sldprt", false, (int)swRebuildOnActivation_e.swUserDecision, ref errors);
            swCompModel = (ModelDoc2)swApp.ActiveDoc;
            swCompModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Sketch9", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swCompModel.EditSketch();
            swCompModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("D3@Sketch9@oleostrut.SLDPRT", "DIMENSION", 0.0792805491990847, -0.020779176201373, 0, false, 0, null, 0);
            swDimension = (Dimension)swCompModel.Parameter("D3@Sketch9");
            swDimension.SystemValue = 0.025;
            swCompModel.ClearSelection2(true);
            status = swCompModel.Save3((int)swSaveAsOptions_e.swSaveAsOptions_SaveReferenced, ref errors, ref warnings);
            swApp.CloseDoc("oleostrut.sldprt");

            // Set the transparency level of unmodified components in the assembly
            swAssemblyDoc.LargeDesignReviewTransparencyLevelDynamic = true;
            swAssemblyDoc.LargeDesignReviewTransparencyLevelEnabled = true;
            swAssemblyDoc.LargeDesignReviewTransparencyLevel = 0.75;
            Debug.Print("Transparency level: " + swAssemblyDoc.LargeDesignReviewTransparencyLevel);
            Debug.Print("    Enabled = " + swAssemblyDoc.LargeDesignReviewTransparencyLevelEnabled);
            Debug.Print("    Dynamic = " + swAssemblyDoc.LargeDesignReviewTransparencyLevelDynamic);
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;
}
```

Set Transparency of Unmodified Components in Large Design Review Mode Example (VBA)

This example shows how to:

- open an assembly in Large Design Review mode
- open, modify, save, and close an assembly component, and
- set the transparency level of unmodified components in the assembly.

```

'-----
' Preconditions:
' 1. Create c:\temp\LDR.
' 2. Copy these files from public_documents\samples\tutorial\api to c:\temp\LDR:
' * landing_gear.sldasm
' * lwrsway_lnk.sldprt
' * oleopiston.sldprt
' * oleostrut.sldprt
' * part4.sldprt
' * upprsway_lnk.sldprt
' * wheel_hub.sldprt
' * wheelassy.sldasm
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Click OK in the Large Design Review dialog.
' 2. Opens the assembly in Large Design Review mode.
' 3. Opens, modifies, saves, and closes the assembly component, oleostrut.sldprt.
'   a. When prompted to rebuild, click Rebuild and save the document.
'   b. If prompted to update the graphics data in Large Design Review,
'      click Yes.
' 4. Sets the transparency level of unmodified components in the assembly to 0.75.
' 5. Close the assembly and part documents.
' 6. Examine the Immediate window and graphics area.
'-----

```

Option Explicit

Sub main()

```

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swAssemblyDoc As SldWorks.AssemblyDoc
    Dim fileName As String
    Dim status As Boolean
    Dim errors As Long, warnings As Long

    Set swApp = Application.SldWorks

    ' Open assembly in Large Design Review mode
    fileName = "C:\temp\LDR\landing_gear.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocASSEMBLY, swOpenDocOptions_ViewOnly, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swAssemblyDoc = swApp.ActiveDoc

    ' Open component, modify it, and save it
    Dim swCompModel As ModelDoc2
    status = swModelDocExt.SelectByID2("oleostrut-1@landing_gear", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    swApp.OpenDoc6 "C:\temp\LDR\oleostrut.sldprt", 1, 0, "", errors, warnings
    swApp.ActivateDoc3 "oleostrut.sldprt", False, swRebuildOnActivation_e.swUserDecision, errors
    Set swCompModel = swApp.ActiveDoc
    swCompModel.ClearSelection2 (True)
    status = swModelDocExt.SelectByID2("Sketch9", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    swCompModel.EditSketch
    swCompModel.ClearSelection2 (True)
    status = swModelDocExt.SelectByID2("D3@Sketch9@oleostrut.SLDPRT", "DIMENSION", 7.92805491990847E-02, -0.020779176201373,
    0, False, 0, Nothing, 0)
    Dim swDimension As Object
    Set swDimension = swCompModel.Parameter("D3@Sketch9")
    swDimension.SystemValue = 0.025
    swCompModel.ClearSelection2 (True)
    status = swCompModel.Save3(swSaveAsOptions_e.swSaveAsOptions_SaveReferenced, errors, warnings)
    swApp.CloseDoc ("oleostrut.sldprt")

    ' Set the transparency level of unmodified components in the assembly
    swAssemblyDoc.LargeDesignReviewTransparencyLevelDynamic = True
    swAssemblyDoc.LargeDesignReviewTransparencyLevelEnabled = True
    swAssemblyDoc.LargeDesignReviewTransparencyLevel = 0.75
    Debug.Print ("Transparency level: " & swAssemblyDoc.LargeDesignReviewTransparencyLevel)
    Debug.Print (" Enabled = " & swAssemblyDoc.LargeDesignReviewTransparencyLevelEnabled)
    Debug.Print (" Dynamic = " & swAssemblyDoc.LargeDesignReviewTransparencyLevelDynamic)

```

End Sub

Set Transparency of Unmodified Components in Large Design Review Assembly Example (VB.NET)

This example shows how to:

- open an assembly in Large Design Review mode
- open, modify, save, and close an assembly component, and
- set the transparency level of unmodified components in the assembly.

```

-----
Preconditions:
1. Create c:\temp\LDR.
2. Copy these files from public_documents\samples\tutorial\api to c:\temp\LDR:
   * landing_gear.sldasm
   * lwrsway_lnk.sldprt
   * oleopiston.sldprt
   * oleostrut.sldprt
   * part4.sldprt
   * upprsway_lnk.sldprt
   * wheel_hub.sldprt
   * wheelassy.sldasm
3. Open the Immediate window.
Postconditions:
1. Click OK in the Large Design Review dialog.
2. Opens the assembly in Large Design Review mode.
3. Opens, modifies, saves, and closes the assembly component, oleostrut.sldprt.
   a. When prompted to rebuild, click Rebuild and save the document.
   b. If prompted to update the graphics data in Large Design Review,
      click Yes.
4. Sets the transparency level of unmodified components in the assembly to 0.75.
5. Close the assembly and part documents.
6. Examine the Immediate window and graphics area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swAssemblyDoc As AssemblyDoc
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer, warnings As Integer

        ' Open assembly in Large Design Review mode
        fileName = "C:\temp\LDR\landing_gear.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_ViewOnly, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swAssemblyDoc = swApp.ActiveDoc

        ' Open component, modify it, and save it
        Dim swCompModel As ModelDoc2
        status = swModelDocExt.SelectByID2("oleostrut-1@landing_gear", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swApp.OpenDoc6("C:\temp\LDR\oleostrut.sldprt", 1, 0, "", errors, warnings)
        swApp.ActivateDoc3("oleostrut.sldprt", False, swRebuildOnActivation_e.swUserDecision, errors)
        swCompModel = swApp.ActiveDoc
        swCompModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Sketch9", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swCompModel.EditSketch()
        swCompModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("D3@Sketch9@oleostrut.SLDPRT", "DIMENSION", 0.0792805491990847, -0.020779176201373, 0, False, 0, Nothing, 0)
        Dim swDimension As Object
        swDimension = swCompModel.Parameter("D3@Sketch9")
        swDimension.SystemValue = 0.025
        swCompModel.ClearSelection2(True)
        status = swCompModel.Save3(swSaveAsOptions_e.swSaveAsOptions_SaveReferenced, errors, warnings)
        swApp.CloseDoc("oleostrut.sldprt")

        ' Set the transparency level of unmodified components in the assembly
        swAssemblyDoc.LargeDesignReviewTransparencyLevelDynamic = True
        swAssemblyDoc.LargeDesignReviewTransparencyLevelEnabled = True
        swAssemblyDoc.LargeDesignReviewTransparencyLevel = 0.75
        Debug.Print("Transparency level: " & swAssemblyDoc.LargeDesignReviewTransparencyLevel)
        Debug.Print(" Enabled = " & swAssemblyDoc.LargeDesignReviewTransparencyLevelEnabled)
        Debug.Print(" Dynamic = " & swAssemblyDoc.LargeDesignReviewTransparencyLevelDynamic)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Change Setup of Drawing Sheet Example (VBA)

This example shows how to change the setup of a drawing sheet by changing its paper size and template.

```
'-----  
' Preconditions:  
' 1. Verify that the specified sheet format template exists.  
' 2. Open public_documents\samples\tutorial\api\2012-sm.slddrw.  
'  
' Postconditions:  
' 1. Changes the sheet format's paper size and template.  
' 2. Examine the drawing.  
'  
' NOTE: Because the drawing is used elsewhere, do not save  
' changes.  
'-----  
Option Explicit  
  
Sub main()  
  
    Const sTemplatePath As String = "C:\ProgramData\SolidWorks\SOLIDWORKS 2016\lang\english\sheetformat\b-landscape.slddrt"  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swDraw As SldWorks.DrawingDoc  
    Dim swSheet As SldWorks.Sheet  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDraw = swModel  
    Set swSheet = swDraw.GetCurrentSheet  
  
    bRet = swDraw.SetupSheet4(swSheet.GetName, swDwgPaperBsize, swDwgTemplateBsize, 1#, 1#, False, "", 0#, 0#, "")  
  
    swModel.ForceRebuild3 (False)  
  
End Sub
```

Set Up PropertyManager Page Slider Control Example (C++ COM)

The following example shows how to set up a slider control.

```
...
id = CONTROL_SLIDER;
controlType = swControlType_Slider;
caption = _T("");
alignment = swControlAlign_Indent;
options = swControlOptions_Visible + swControlOptions_Enabled;
tip = _T("Second trackbar");
hres = m_pSwGroup_1->IAddControl(id, controlType, caption, alignment, options, tip, &m_pSwControl_1_2);
if (m_pSwControl_1_2 != NULL)
{
    IPropertyManagerPageSlider* pSlider = NULL;
    HRESULT res = m_pSwControl_1_2->QueryInterface(IID_IPropertyManagerPageSlider, (LPVOID*)&pSlider);
    if (pSlider != NULL)
    {
        res = pSlider->put_Style(swPropMgrPageSliderStyle_Vertical | swPropMgrPageSliderStyle_AutoTicks | swPropMgrPageSliderStyle_BottomLeftTicks | swPropMgrPageSliderStyle_NotifyWhileTracking);
        res = pSlider->SetRange(100, 200, &boolstatus);
        res = pSlider->put_Position(125);
        res = pSlider->put_LineSize(2);
        res = pSlider->put_PageSize(10);
        res = pSlider->put_Height(250);

        pSlider->Release();
    }
}
...
```

Set View Display Mode Example (C++ COM)

This example shows how to set the display mode of drawing views.

```
STDMETHODIMP CDgtlPen::demo2()
```

```
{
    HRESULT                hr = S_OK;
    CComPtr <IModelDoc2>    pModel;
    CComQIPtr <IDrawingDoc> pDraw;
    CComPtr <ISheet>        pSheet;
    CComPtr <IView>         pSheetView;
    CComPtr <IView>         pView;
    long                   swDisplayMode = -1;
    VARIANT_BOOL           UseParent = FALSE;
    VARIANT_BOOL           Facetted = TRUE;
    VARIANT_BOOL           Edges = TRUE;
    VARIANT_BOOL           retval = NULL;
    hr = m_pSldWorks->get_IActiveDoc2(&pModel);
    _ASSERT(pModel);
    pDraw = pModel;
    _ASSERT(pDraw);
    hr = pDraw->IGetCurrentSheet(&pSheet);
    _ASSERT(pSheet);
    hr = pDraw->IGetFirstView(&pSheetView);
    _ASSERT(pSheetView);
    hr = pSheetView->IGetNextView(&pView);
    _ASSERT(pView);
    while(pView != NULL)
    {
        CComPtr <IView>      pNextView;
        hr = pView->GetDisplayMode2(&swDisplayMode);
        switch(swDisplayMode)
        {
            case swWIREFRAME:
            case swHIDDEN_GREYED:
            case swHIDDEN:
                break;
            case swSHADED:
            case swFACETED_WIREFRAME:
            case swFACETED_HIDDEN_GREYED:
            case swFACETED_HIDDEN:
                hr = pView->SetDisplayMode3(UseParent, swHIDDEN, Facetted, Edges, &retval);
                _ASSERT(retval);
                break;
            default :
                _ASSERT(FALSE);
                break;
        }
        hr = pView->IGetNextView(&pNextView);
        // Release view
        pView = NULL;
        pView = pNextView;
    }
    return S_OK;
}
```

Set View Scale Example (VBA)

This example shows how to set the scale of a selected drawing view.

```
'-----
' Preconditions:
' 1. Open public_documents\introsw\bolt-assembly.slddrw.
' 2. Select a drawing view.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Increases the selected drawing view's scale.
' 2. Examine the drawing and Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not
' save changes.
'-----
```

Option Explicit

```
Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swView As SldWorks.View
    Dim vScaleRatio As Variant
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)

    vScaleRatio = swView.ScaleRatio

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " View = " & swView.Name
    Debug.Print " Use sheet scale = " & CBool(swView.UseSheetScale)
    Debug.Print " Original scale ratio = " & vScaleRatio(0) & ":" & vScaleRatio(1)
    Debug.Print " Original decimal scale value = " & swView.ScaleDecimal

    ' Increase scale values
    ' Changing scale sets IView::UseSheetScale to false
    vScaleRatio = swView.ScaleRatio
    swView.ScaleDecimal = swView.ScaleDecimal * 2#
    vScaleRatio = swView.ScaleRatio
    Debug.Print ""
    Debug.Print " Use sheet scale = " & CBool(swView.UseSheetScale)
    Debug.Print " New scale ratio = " & vScaleRatio(0) & ":" & vScaleRatio(1)
    Debug.Print " New decimal scale value = " & swView.ScaleDecimal

    ' Rebuild to see the scaled drawing view
    bRet = swModel.EditRebuild3: Debug.Assert bRet

End Sub
```

Set View Scale Opposite Parent View Scale Example (VBA)

This example shows how to set the selected drawing view's scale opposite the parent's drawing view's scale.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
' 2. Select Drawing View2.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Sets Drawing View2's scale opposite
'   of the parent's drawing's view scale.
' 2. Examine the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save
' changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swDraw As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " View = " & swView.Name
    Debug.Print " Original UseParentScale = " & CBool(swView.UseParentScale)

    ' Toggle
    If swView.UseParentScale Then
        swView.UseParentScale = False
    Else
        swView.UseParentScale = True
    End If

    Debug.Print " Toggled UseParentScale = " & CBool(swView.UseParentScale)

    ' Rebuild to see new scale
    bRet = swModel.EditRebuild3: Debug.Assert bRet

End Su
```